ĐẠI HỌC ĐÀ NẴNG

TRƯỜNG ĐẠI HỌC BÁCH KHOA



LG DUT CYBER SECURITY LAB

BÁO CÁO THỰC TẬP TUẦN 4

Nghiên cứu và triển khai `ulogd2` trên WebOS phục vụ ghi nhận nhật ký Netfilter

Thành viên Nhóm 4

1. Nguyễn Trường Bảo
2. Mai Thị Mỹ Đình
3. Bùi Phương Chi
4. Nguyễn Đình Nhất
5. Đỗ Hữu Dũng
6. Nguyễn Thành Lưu

Đà Nẵng, ngày ... tháng ... năm ...

Mục lục

LỜI MỞ ĐẦU **1**

1 GIỚI THIỆU VÀ MỤC TIÊU **3**

1.1 Bối cảnh . . . 3
1.2 Đặt vấn đề . . . 3
1.3 Mục tiêu . . . 4
1.4 Phạm vi thực hiện . . . 4
1.5 Kiến trúc tổng quan của hệ thống . . . 5
1.6 Cấu trúc báo cáo . . . 5

2 KIẾN TRÚC ULOGD2 **7**

2.1 Kiến trúc tổng thể của ulogd2 . . . 7
2.1.1 Giới thiệu . . . 7
2.1.2 Mô hình Pipeline . . . 7
2.1.3 Đặc điểm của kiến trúc Pipeline . . . 9
2.2 Cơ chế Plugin của ulogd2 . . . 9
2.2.1 Quá trình nạp Plugin . . . 9
2.2.2 Cấu hình Pipeline . . . 10
2.2.3 Khả năng mở rộng . . . 10
2.3 Luồng hoạt động của ulogd2 . . . 11

3 NGHIÊN CỨU CÁC INPUT PLUGINS (ULOG, NFLOG, NFCT) **13**

3.1 Input Plugin ULOG . . . 13
3.1.1 Giới thiệu . . . 13
3.1.2 Ưu điểm . . . 13
3.1.3 Nhược điểm . . . 13
3.1.4 Trường hợp sử dụng . . . 13
3.2 Input Plugin NFLOG . . . 14
3.2.1 Giới thiệu . . . 14
3.2.2 Quy trình hoạt động . . . 14
3.2.3 Các tham số của NFLOG . . . 14
3.2.4 Ví dụ cấu hình NFLOG . . . 15
3.2.5 Ưu điểm . . . 15

3.2.6 Nhược điểm . . . 15
3.2.7 Trường hợp sử dụng . . . 15
3.3 Input Plugin NFCT . . . 16
3.3.1 Giới thiệu . . . 16
3.3.2 Kiến trúc hoạt động . . . 16
3.3.3 Các trạng thái của Connection Tracking . . . 16
3.3.4 Ưu điểm . . . 17
3.3.5 Nhược điểm . . . 17
3.3.6 Trường hợp sử dụng . . . 17
3.4 So sánh các Input Plugin . . . 17

4 NGHIÊN CỨU CÁC OUTPUT PLUGINS 19

4.1 Tổng quan về Output Plugin . . . 19
4.2 Output Plugin LOGEMU . . . 19
4.2.1 Giới thiệu . . . 19
4.2.2 Nguyên lý hoạt động . . . 19
4.2.3 Ưu điểm . . . 19
4.2.4 Nhược điểm . . . 19
4.2.5 Trường hợp sử dụng . . . 19
4.3 Output Plugin OPRINT . . . 19
4.3.1 Giới thiệu . . . 20
4.3.2 Nguyên lý hoạt động . . . 20
4.3.3 Ưu điểm . . . 20
4.3.4 Nhược điểm . . . 20
4.3.5 Trường hợp sử dụng . . . 20
4.4 Output Plugin SYSLOG . . . 20
4.4.1 Giới thiệu . . . 20
4.4.2 Nguyên lý hoạt động . . . 20
4.4.3 Ưu điểm . . . 20
4.4.4 Nhược điểm . . . 20
4.4.5 Trường hợp sử dụng . . . 21
4.5 Output Plugin MYSQL . . . 21
4.5.1 Giới thiệu . . . 21
4.5.2 Nguyên lý hoạt động . . . 21

4.5.3 Ưu điểm . . . 21
4.5.4 Nhược điểm . . . 21
4.5.5 Trường hợp sử dụng . . . 21
4.6 Output Plugin PGSQL . . . 21
4.6.1 Giới thiệu . . . 21
4.6.2 Nguyên lý hoạt động . . . 21
4.6.3 Ưu điểm . . . 21
4.6.4 Nhược điểm . . . 22
4.6.5 Trường hợp sử dụng . . . 22
4.7 Output Plugin SQLITE3 . . . 22
4.7.1 Giới thiệu . . . 22
4.7.2 Nguyên lý hoạt động . . . 22
4.7.3 Ưu điểm . . . 22
4.7.4 Nhược điểm . . . 22
4.7.5 Trường hợp sử dụng . . . 22
4.8 Output Plugin PCAP . . . 22
4.8.1 Giới thiệu . . . 22
4.8.2 Nguyên lý hoạt động . . . 22
4.8.3 Ưu điểm . . . 23
4.8.4 Nhược điểm . . . 23
4.8.5 Trường hợp sử dụng . . . 23
4.9 Output Plugin IPFIX . . . 23
4.9.1 Giới thiệu . . . 23
4.9.2 Nguyên lý hoạt động . . . 23
4.9.3 Ưu điểm . . . 23
4.9.4 Nhược điểm . . . 23
4.9.5 Trường hợp sử dụng . . . 23
4.10 Output Plugin NACCT . . . 23
4.10.1 Giới thiệu . . . 23
4.10.2 Nguyên lý hoạt động . . . 24
4.10.3 Ưu điểm . . . 24
4.10.4 Nhược điểm . . . 24
4.10.5 Trường hợp sử dụng . . . 24

4.11 Output Plugin JSON . . . 24
4.11.1 Giới thiệu . . . 24
4.11.2 Nguyên lý hoạt động . . . 24
4.11.3 Ưu điểm . . . 24
4.11.4 Nhược điểm . . . 24
4.11.5 Trường hợp sử dụng . . . 25
4.12 So sánh các Output Plugin . . . 25

5 KHẢO SÁT ULOGD2 TRÊN WEBOS 26
5.1 Giới thiệu . . . 26
5.2 Tích hợp recipe ulogd2 vào hệ thống build . . . 26
5.3 Kiểm tra kết quả build . . . 27
5.4 Kiểm tra phiên bản ulogd2 trên thiết bị . . . 27
5.5 Kiểm tra các Plugin của ulogd2 . . . 28
5.6 Danh sách Plugin được hỗ trợ . . . 28
5.6.1 Input Plugin . . . 29
5.6.2 Filter Plugin . . . 29
5.6.3 Output Plugin . . . 30
5.7 Đánh giá khả năng hỗ trợ . . . 30

6 XÂY DỰNG ULOGD TÙY BIẾN VÀ TRIỂN KHAI THỰC TẾ 32
6.1 Yêu cầu của VanFit và hỗ trợ xuất JSON (libjansson) . . . 32
6.2 Nguyên tắc đặt vị trí rule NFLOG . . . 32
6.3 Bảng tổng hợp NFLOG group . . . 33
6.4 Cấu hình /etc/ulogd.conf hoàn chỉnh . . . 33
6.5 Chi tiết các Test Case . . . 35
6.5.1 TC-W4-01 — Log ICMP Ping IPv4 . . . 35
6.5.2 TC-W4-02 — Log SYN Flood attack . . . 38
6.5.3 TC-W4-03 — Log Broadcast IPv4 . . . 39
6.5.4 TC-W4-04 — Log Multicast IPv4 . . . 42
6.5.5 TC-W4-05 — Log Smurf Attack . . . 44
6.5.6 TC-W4-06 — Audit traffic OUTPUT hợp lệ . . . 46

7 XỬ LÝ SỰ CỐ VÀ QUYẾT ĐỊNH THIẾT KẾ 49
7.1 Xung đột cơ chế lắng nghe của NFLOG (Lỗi rỗng file JSON & Unable to bind) . . . 49

7.2 Thiếu trường dữ liệu phần cứng (Lỗi cannot find key `oob.in`) 49
7.3 Điều chỉnh kịch bản kiểm thử: thay Port Scanning/LAND bằng Broadcast/Multicast . . 49

8 KẾT LUẬN VÀ BÀI HỌC KINH NGHIỆM 51

8.1 Tổng kết thành quả triển khai 51
8.2 Nguyên lý cốt lõi rút ra (Core Principles) 51
8.3 Nhận xét cá nhân và định hướng phát triển 51

TÀI LIỆU THAM KHẢO 52

LỜI MỞ ĐẦU

Trong những năm gần đây, cùng với sự phát triển mạnh mẽ của Internet of Things (IoT), hệ thống nhúng và ngành công nghiệp ô tô thông minh (Automotive), các thiết bị chạy hệ điều hành Linux ngày càng được ứng dụng rộng rãi trong nhiều lĩnh vực. Đặc biệt, nền tảng WebOS Automotive được sử dụng trên các hệ thống giải trí và điều khiển trong xe, nơi các thành phần phần mềm thường xuyên trao đổi dữ liệu thông qua mạng nội bộ và kết nối với nhiều dịch vụ bên ngoài. Điều này đặt ra yêu cầu ngày càng cao về khả năng giám sát, ghi nhận và phân tích các hoạt động mạng nhằm đảm bảo tính ổn định, an toàn và bảo mật của toàn bộ hệ thống.

Trong lĩnh vực an toàn thông tin, việc thu thập và lưu trữ nhật ký (logging) đóng vai trò vô cùng quan trọng. Các bản ghi không chỉ phản ánh trạng thái hoạt động của hệ thống tại từng thời điểm mà còn cung cấp những bằng chứng cần thiết phục vụ cho quá trình điều tra sự cố, kiểm toán bảo mật, phát hiện hành vi bất thường cũng như hỗ trợ xây dựng các hệ thống giám sát tập trung (SIEM). Đối với các hệ thống nhúng trên ô tô, nơi yêu cầu độ tin cậy và tính an toàn rất cao, việc ghi nhận đầy đủ các sự kiện liên quan đến lưu lượng mạng là một trong những yếu tố quan trọng giúp phát hiện sớm các nguy cơ tấn công, sai lệch cấu hình hoặc các hành vi truy cập trái phép.

Linux Kernel cung cấp framework Netfilter cùng công cụ iptables để thực hiện chức năng lọc gói tin, kiểm soát truy cập mạng và áp dụng các chính sách bảo mật. Bên cạnh đó, iptables cũng hỗ trợ ghi nhận các gói tin thông qua các target như LOG, ULOG và NFLOG. Trong đó, target LOG là phương pháp ghi log truyền thống bằng cách gửi thông tin trực tiếp tới kernel log hoặc syslog. Tuy nhiên, phương pháp này còn tồn tại nhiều hạn chế như dữ liệu ghi nhận chưa có cấu trúc rõ ràng, khó mở rộng, khó tích hợp với các hệ thống phân tích hiện đại, đồng thời không hỗ trợ linh hoạt nhiều định dạng đầu ra phục vụ các nhu cầu lưu trữ và xử lý khác nhau.

Để khắc phục những hạn chế trên, ulogd2 (Userspace Logging Daemon version 2) được phát triển như một trình nền hoạt động ở không gian người dùng (user space), có nhiệm vụ tiếp nhận thông tin từ Netfilter thông qua cơ chế Netlink và xử lý dữ liệu bằng kiến trúc Plugin. Thay vì chỉ ghi log theo một định dạng cố định, ulogd2 cho phép người quản trị xây dựng các pipeline xử lý linh hoạt bao gồm các Input Plugin, Filter Plugin và Output Plugin. Nhờ đó, dữ liệu có thể được chuyển đổi, bổ sung thông tin và xuất ra nhiều định dạng khác nhau như LOGEMU, SYSLOG, JSON, SQLite, MySQL, PostgreSQL, PCAP hay IPFIX, đáp ứng các nhu cầu từ lưu trữ cục bộ đến tích hợp với các hệ thống giám sát và phân tích tập trung.

Trong khuôn khổ tuần thực tập này, nội dung nghiên cứu tập trung vào việc tìm hiểu kiến trúc hoạt động của ulogd2 trên nền tảng WebOS, khảo sát các plugin đầu vào (Input Plugin) và đầu ra (Output Plugin) được hệ thống hỗ trợ, đồng thời đánh giá vai trò và nguyên lý hoạt động của từng plugin trong quá trình xử lý dữ liệu mạng. Bên cạnh đó, báo cáo cũng tiến hành xây dựng hệ thống ghi log tùy biến bằng cách cấu hình NFLOG làm nguồn dữ liệu đầu vào, kết hợp với JSON Output Plugin để xuất báo cáo theo định dạng JSON và LOGEMU để ghi nhận log dạng văn bản phục vụ kiểm tra trực tiếp.

Ngoài việc cấu hình ulogd2, báo cáo còn xây dựng các chính sách bảo mật trên iptables nhằm sinh ra các sự kiện mạng cần giám sát, cấu hình các nhóm NFLOG tương ứng với từng loại lưu lượng, đồng thời kiểm tra quá trình thu thập và xuất dữ liệu thông qua các tệp full.log và firewall_log.json. Đối với yêu cầu triển khai của doanh nghiệp giả định VanFit, hệ thống còn được mở rộng bằng cách bổ sung thư viện libjansson và điều chỉnh quá trình biên dịch ulogd2 để hỗ trợ xuất dữ liệu theo định

dạng JSON, tạo tiền đề cho việc tích hợp với các hệ thống phân tích log hoặc các nền tảng AI phục vụ phát hiện bất thường trong tương lai.

Thông qua quá trình nghiên cứu và triển khai thực tế trên Raspberry Pi chạy WebOS, báo cáo không chỉ giúp làm rõ nguyên lý hoạt động của ulogd2 mà còn đánh giá khả năng ứng dụng của hệ thống ghi log này trong việc xây dựng các giải pháp giám sát mạng, kiểm toán an toàn thông tin và quản lý sự kiện bảo mật dành cho các hệ thống Linux nhúng nói chung và WebOS Automotive nói riêng.

1 GIỚI THIỆU VÀ MỤC TIÊU

1.1 Bối cảnh

Trong các hệ điều hành Linux hiện đại, Netfilter là thành phần cốt lõi chịu trách nhiệm xử lý, kiểm soát và lọc các gói tin mạng ở mức kernel. Thông qua các công cụ như iptables hoặc nftables, người quản trị có thể xây dựng các chính sách bảo mật nhằm kiểm soát luồng dữ liệu ra vào hệ thống, bao gồm cho phép (ACCEPT), từ chối (DROP), ghi nhận (LOG) hoặc chuyển tiếp (FORWARD) các gói tin.

Trong các hệ thống nhúng (Embedded Linux) như webOS Open Source Edition (webOS OSE), việc giám sát hoạt động mạng có vai trò đặc biệt quan trọng. Thiết bị cần có khả năng ghi nhận các sự kiện liên quan đến lưu lượng mạng để phục vụ nhiều mục đích như:

- Giám sát an toàn thông tin (Security Monitoring).
- Kiểm toán hệ thống (Security Auditing).
- Phân tích sự cố (Troubleshooting).
- Điều tra số (Digital Forensics).
- Thu thập dữ liệu phục vụ phân tích hành vi mạng.

Mặc dù iptables hỗ trợ mục tiêu LOG để ghi nhận thông tin gói tin, phương pháp này chủ yếu ghi log vào kernel log (syslog hoặc dmesg), gây ra nhiều hạn chế:

- Khó tùy biến nội dung log.
- Khó xử lý lưu lượng lớn.
- Không hỗ trợ xuất dữ liệu theo nhiều định dạng.
- Không phù hợp với các hệ thống cần phân tích dữ liệu hoặc tích hợp với các nền tảng giám sát.

Để khắc phục các hạn chế trên, dự án Netfilter phát triển ulogd2 (Userspace Logging Daemon version 2). Đây là một daemon hoạt động trong không gian người dùng (userspace), có khả năng tiếp nhận thông tin từ Netfilter thông qua nhiều cơ chế khác nhau như ULOG, NFLOG và NFCT, sau đó xử lý và xuất dữ liệu dưới nhiều định dạng khác nhau như LOGEMU, SYSLOG, JSON, SQLite, PCAP, IPFIX, SQL Database, NACCT. Nhờ kiến trúc plugin, ulogd2 cho phép người sử dụng xây dựng các pipeline xử lý dữ liệu linh hoạt, đáp ứng nhiều yêu cầu khác nhau về ghi log, kiểm toán và phân tích lưu lượng mạng.

1.2 Đặt vấn đề

Trong chương trình thực hành tuần 4, yêu cầu đặt ra là tích hợp ulogd2 vào hệ điều hành webOS OSE chạy trên Raspberry Pi, đồng thời cấu hình hệ thống để ghi nhận các sự kiện Netfilter theo định dạng JSON.

Tuy nhiên, sau khi khảo sát môi trường build webOS OSE phiên bản 2.20, nhận thấy hệ thống chưa tích hợp sẵn recipe của ulogd2. Khi thực hiện lệnh:

```
bitbake ulogd2
```

BitBake trả về lỗi:

```
ERROR: Nothing PROVIDES 'ulogd2'
```

Điều này cho thấy package ulogd2 chưa tồn tại trong các layer hiện có của dự án. Bên cạnh đó, sau khi kiểm tra trực tiếp trên Raspberry Pi, hệ thống cũng không tồn tại chương trình ulogd, file cấu hình ulogd.conf, các plugin Input/Output và service ulogd.service.

Do đó, hệ thống chưa thể đáp ứng yêu cầu ghi log thông qua NFLOG, cũng như chưa thể xuất báo cáo theo định dạng JSON như yêu cầu của bài thực hành. Để giải quyết vấn đề này, cần tiến hành tích hợp recipe ulogd2 từ dự án OpenEmbedded, điều chỉnh recipe để tương thích với môi trường Yocto Kirkstone mà webOS OSE đang sử dụng, sau đó build lại image và triển khai trên Raspberry Pi.

1.3 Mục tiêu

Mục tiêu của đề tài là tích hợp thành công ulogd2 vào hệ điều hành webOS OSE, xây dựng nền tảng phục vụ ghi nhận và kiểm toán lưu lượng mạng thông qua Netfilter. Các mục tiêu cụ thể bao gồm:

- Khảo sát khả năng hỗ trợ ulogd2 của webOS OSE.
- Tích hợp recipe ulogd2 từ OpenEmbedded vào hệ thống build.
- Điều chỉnh recipe để tương thích với môi trường Yocto Kirkstone.
- Build thành công package ulogd2 và xác nhận các plugin Input, Filter, Output.
- Cấu hình NFLOG làm nguồn đầu vào; xuất log song song LOGEMU (full.log) và JSON (firewall_log.json).
- Xây dựng và kiểm thử các test case gắn với cơ chế phòng thủ các tuần trước (DDoS, VLAN/IPv6, Unicast/Multicast/Broadcast).
- Ghi nhận bằng chứng (audit log) cho cả traffic bị chặn và traffic hợp lệ được chấp nhận.

1.4 Phạm vi thực hiện

Đề tài tập trung vào quá trình tích hợp, cấu hình và kiểm thử ulogd2 trên nền tảng webOS OSE. Các nội dung thực hiện bao gồm:

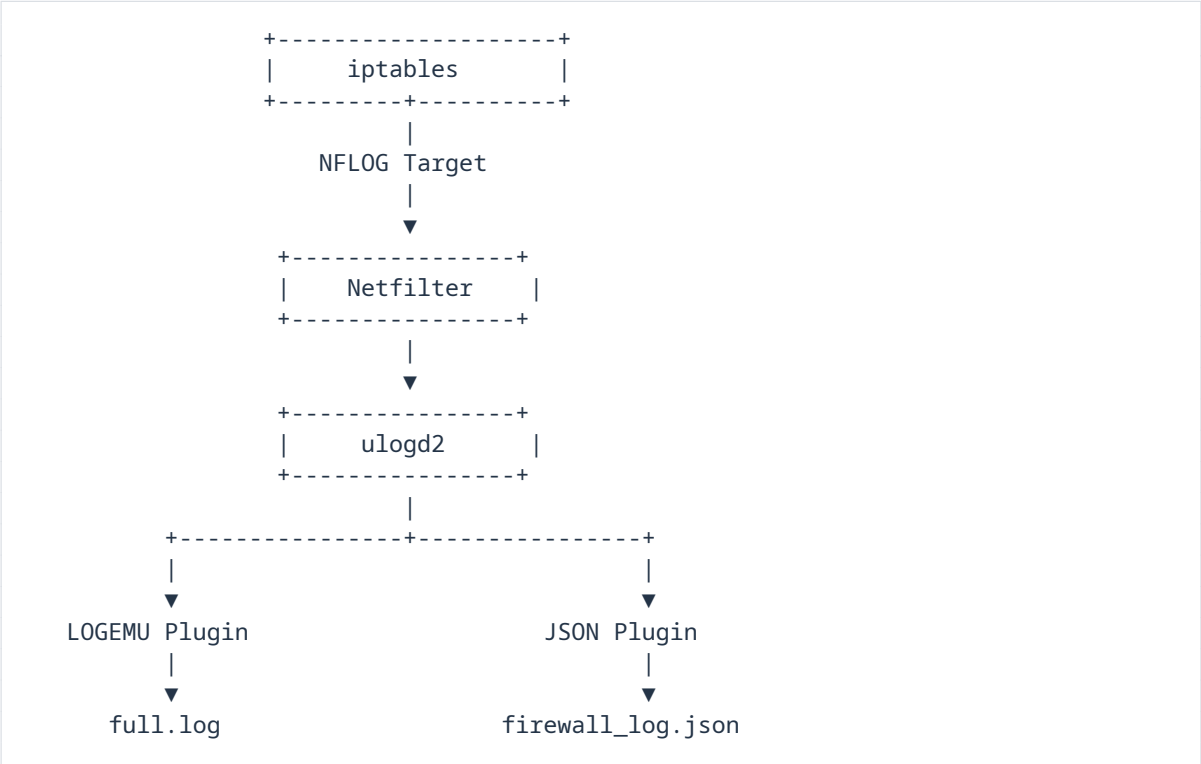
- Khảo sát hệ thống build webOS OSE và tích hợp recipe ulogd2.
- Khảo sát plugin trên thiết bị sau khi nạp image.
- Cấu hình ulogd.conf, rule iptables/ip6tables với NFLOG.
- Triển khai test case và phân tích log thực tế.
- Ghi nhận xử lý sự cố và quyết định thiết kế.

Đề tài không tập trung vào:

- Xây dựng hệ thống SIEM hoàn chỉnh.
- Phân tích log trên Elasticsearch hoặc Splunk ở quy mô sản xuất.
- Tối ưu hiệu năng ghi log và đánh giá chịu tải trong môi trường production.

1.5 Kiến trúc tổng quan của hệ thống

Sau khi được tích hợp vào webOS OSE, quá trình ghi nhận dữ liệu mạng dự kiến hoạt động theo luồng sau:



Trong mô hình trên:

- iptables chịu trách nhiệm xác định các gói tin cần ghi nhận.
- NFLOG chuyển các gói tin từ kernel sang userspace.
- ulogd2 tiếp nhận dữ liệu, xử lý thông qua các plugin.
- LOGEMU ghi log dưới dạng văn bản.
- JSON Plugin xuất dữ liệu theo định dạng JSON để phục vụ phân tích hoặc tích hợp với các hệ thống giám sát.

Kiến trúc này cho phép mở rộng linh hoạt bằng cách bổ sung các plugin khác như SYSLOG, SQLite, PCAP hoặc IPFIX mà không cần thay đổi các rule của Netfilter.

1.6 Cấu trúc báo cáo

Báo cáo được tổ chức theo các chương như sau:

- Chương 1: Giới thiệu và mục tiêu.
- Chương 2: Kiến trúc ulogd2 (Pipeline, Plugin, luồng hoạt động).
- Chương 3: Nghiên cứu các Input Plugin (ULOG, NFLOG, NFCT).
- Chương 4: Nghiên cứu các Output Plugin.
- Chương 5: Khảo sát ulogd2 trên WebOS (tích hợp recipe và kiểm tra plugin trên thiết bị).

- Chương 6: Xây dựng ulogd tùy biến và triển khai thực tế (VanFit/JSON, cấu hình, test case).
- Chương 7: Xử lý sự cố và quyết định thiết kế.
- Chương 8: Kết luận và bài học kinh nghiệm.
- Tài liệu tham khảo.

2 KIẾN TRÚC ULOGD2

2.1 Kiến trúc tổng thể của ulogd2

2.1.1 Giới thiệu

ulogd2 (Userspace Logging Daemon version 2) là trình nền (daemon) hoạt động trong không gian người dùng (User Space), được phát triển nhằm mở rộng khả năng ghi nhận và xử lý nhật ký của Netfilter. Không giống như cơ chế ghi log truyền thống của iptables chỉ ghi trực tiếp lên Kernel Log, ulogd2 tiếp nhận dữ liệu từ Netfilter thông qua giao thức Netlink, sau đó xử lý và xuất dữ liệu theo nhiều định dạng khác nhau.

Điểm nổi bật của ulogd2 là kiến trúc Plugin-Based, trong đó toàn bộ quá trình xử lý dữ liệu được chia thành các module độc lập. Mỗi module đảm nhiệm một chức năng riêng và có thể được thay thế hoặc kết hợp linh hoạt tùy theo nhu cầu của hệ thống. Nhờ kiến trúc này, người quản trị có thể xây dựng các chuỗi xử lý (Pipeline) khác nhau mà không cần sửa đổi mã nguồn của ulogd2.

2.1.2 Mô hình Pipeline

Quá trình xử lý của ulogd2 được xây dựng theo mô hình Pipeline gồm ba nhóm plugin chính. Mỗi nhóm plugin đảm nhiệm một giai đoạn khác nhau trong quá trình xử lý dữ liệu.

a. Input Plugin

Input Plugin là điểm bắt đầu của Pipeline. Nhiệm vụ chính của Input Plugin là thu thập dữ liệu từ các nguồn khác nhau trong Linux Kernel và chuyển dữ liệu này vào ulogd2 để xử lý. Một số Input Plugin phổ biến gồm ULOG, NFLOG và NFCT.

Trong phạm vi báo cáo này, plugin NFLOG được sử dụng làm nguồn dữ liệu đầu vào. Plugin này nhận các gói tin được chuyển từ Netfilter thông qua Netlink Socket sau khi các luật iptables sử dụng target NFLOG được kích hoạt.

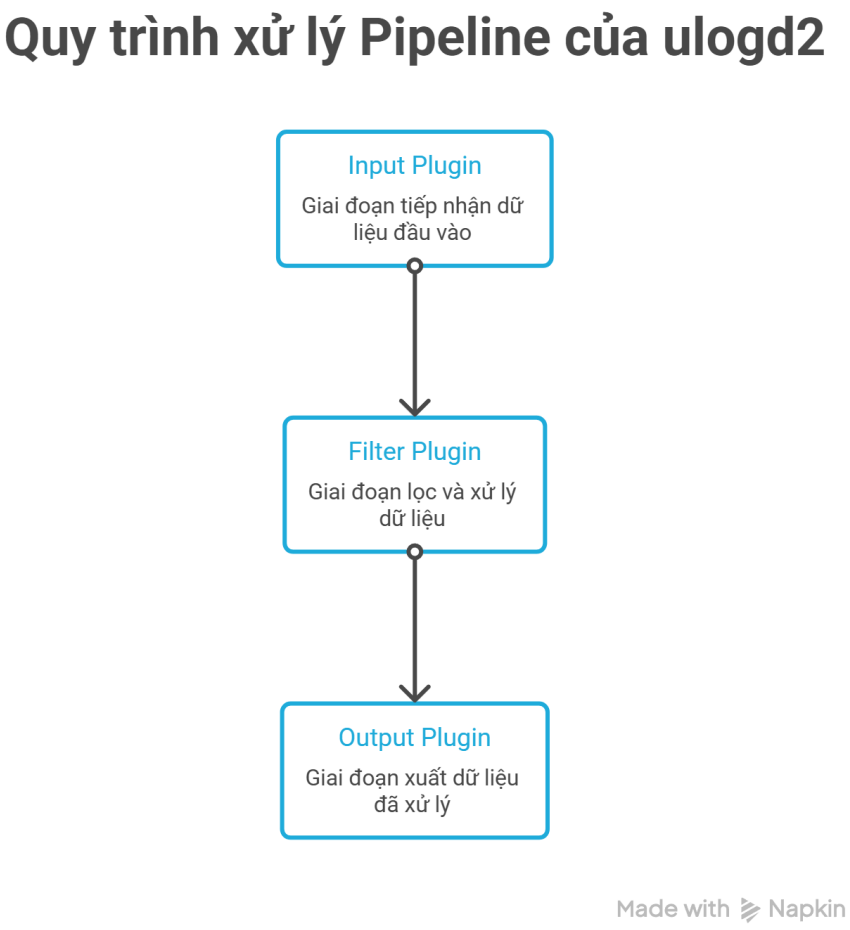
b. Filter Plugin

Sau khi nhận dữ liệu từ Input Plugin, các Filter Plugin sẽ tiến hành xử lý trước khi dữ liệu được ghi ra ngoài. Tùy từng plugin, dữ liệu có thể được:

- Chuyển đổi địa chỉ IP từ dạng nhị phân sang chuỗi ký tự.
- Bổ sung thông tin về Interface.
- Chuyển đổi Timestamp.
- Tính toán độ dài gói tin.
- Trích xuất các trường Header.
- Định dạng dữ liệu.

Một số Filter Plugin thường gặp: BASE, IFINDEX, IP2STR, PRINTPKT, HWHDR. Các Filter Plugin có thể được ghép nối với nhau để tạo thành nhiều Pipeline khác nhau.

c. Output Plugin



Hình 2.1: Mô hình Pipeline của ulogd2

Output Plugin là thành phần cuối cùng trong Pipeline. Sau khi dữ liệu đã được xử lý hoàn chỉnh, Output Plugin sẽ ghi dữ liệu ra các hệ thống lưu trữ hoặc các định dạng mong muốn. ulogd2 hỗ trợ nhiều Output Plugin như LOGEMU, SYSLOG, JSON, SQLITE3, MYSQL, PGSQL, PCAP và IPFIX tùy theo nhu cầu triển khai.

Trong báo cáo này sử dụng đồng thời:

- LOGEMU để ghi dữ liệu dạng văn bản vào file full.log.
- JSON để xuất dữ liệu có cấu trúc vào file firewall_log.json.

2.1.3 Đặc điểm của kiến trúc Pipeline

Việc thiết kế theo mô hình Pipeline mang lại nhiều ưu điểm:

- Các Plugin hoạt động độc lập với nhau.
- Có thể thay đổi Output mà không cần sửa Input.
- Có thể bổ sung nhiều Filter Plugin để xử lý dữ liệu.
- Dễ mở rộng khi cần tích hợp thêm các Plugin mới.
- Giảm sự phụ thuộc giữa các thành phần của hệ thống.

Nhờ vậy, ulogd2 có thể đáp ứng nhiều mục đích sử dụng khác nhau như ghi log Firewall, lưu lượng mạng, Connection Tracking hoặc tích hợp với các hệ thống SIEM.

2.2 Cơ chế Plugin của ulogd2

Một trong những điểm mạnh của ulogd2 là sử dụng kiến trúc Plugin thay vì xây dựng toàn bộ chức năng trong một chương trình duy nhất. Mỗi Plugin được biên dịch thành một thư viện động (Shared Library) có phần mở rộng `.so`.

Ví dụ:

```
libulogd_inppkt_NFLOG.so
libulogd_filter_IP2STR.so
libulogd_filter_IFINDEX.so
libulogd_output_LOGEMU.so
libulogd_output_JSON.so
```

Các thư viện này thường được lưu trong thư mục plugin của ulogd2 và sẽ được nạp (load) khi dịch vụ ulogd2 khởi động.

2.2.1 Quá trình nạp Plugin

Khi ulogd2 được khởi động, chương trình sẽ thực hiện các bước sau:

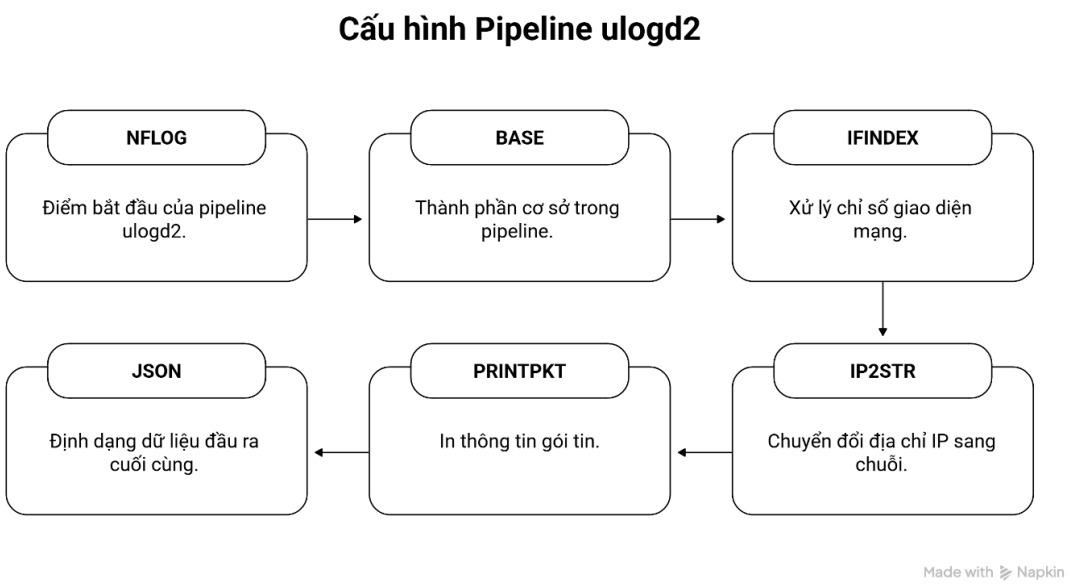
- Đọc file cấu hình ulogd.conf.
- Xác định danh sách Plugin cần sử dụng.
- Nạp các thư viện `.so` tương ứng.

- Khởi tạo từng Plugin.
- Xây dựng Pipeline theo cấu hình.
- Chờ dữ liệu từ Netfilter.

Quá trình này giúp người quản trị có thể thay đổi chức năng của hệ thống chỉ bằng cách chỉnh sửa file cấu hình mà không cần biên dịch lại chương trình.

2.2.2 Cấu hình Pipeline

Pipeline của ulogd2 được định nghĩa trong file cấu hình. Ví dụ một Pipeline đơn giản:



Hình 2.2: Ví dụ cấu hình Pipeline trong ulogd.conf

Trong đó:

- NFLOG tiếp nhận gói tin từ Netfilter.
- BASE trích xuất các thông tin cơ bản.
- IFINDEX chuyển đổi chỉ số Interface thành tên Interface.
- IP2STR chuyển địa chỉ IP sang dạng chuỗi.
- PRINTPKT định dạng dữ liệu.
- JSON ghi dữ liệu thành file JSON.

Người quản trị hoàn toàn có thể thay thế Output Plugin hoặc thêm các Filter Plugin khác tùy theo yêu cầu của hệ thống.

2.2.3 Khả năng mở rộng

Do các Plugin hoạt động độc lập nên việc mở rộng ulogd2 khá đơn giản. Ví dụ:

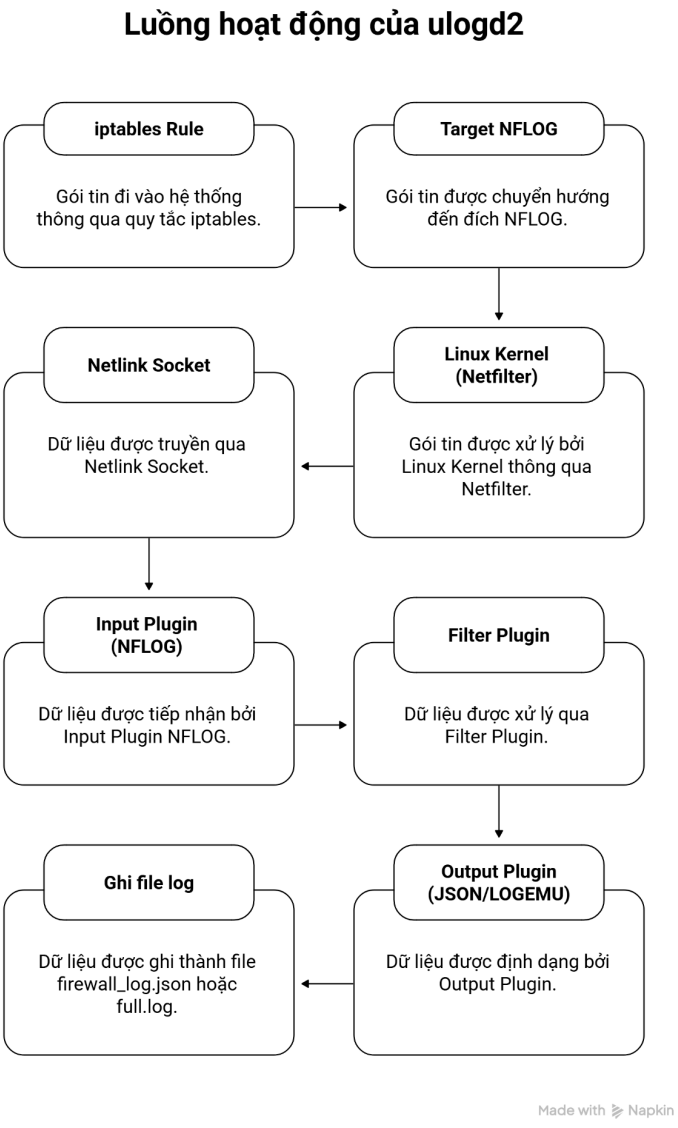
- Muốn lưu vào SQLite chỉ cần thay Output Plugin.

- Muốn lưu vào MySQL chỉ cần thay Plugin Database.
- Muốn lưu dưới dạng PCAP chỉ cần thêm Plugin PCAP.

Không cần thay đổi Input Plugin hoặc các Filter Plugin đang sử dụng. Điều này giúp ulogd2 có khả năng mở rộng cao và dễ dàng tích hợp với nhiều hệ thống phân tích log khác nhau.

2.3 Luồng hoạt động của ulogd2

Để hiểu rõ cách ulogd2 hoạt động, cần xem xét toàn bộ quá trình từ khi một gói tin đi vào hệ thống cho đến khi dữ liệu được ghi thành file. Sơ đồ tổng quát như sau:



Hình 2.3: Luồng hoạt động từ iptables đến ulogd2

Quá trình này bao gồm các bước sau:

Bước 1. Gói tin khớp với luật iptables

Khi một gói tin đi qua Netfilter, iptables sẽ kiểm tra lần lượt các Rule đã được cấu hình. Nếu một Rule sử dụng target NFLOG, gói tin sẽ không bị ghi trực tiếp vào Kernel Log mà được chuyển đến hệ thống NFLOG. Ví dụ:

```
iptables -A INPUT -j NFLOG --nflog-group 169
```

Bước 2. Netfilter chuyển dữ liệu qua Netlink

Sau khi Rule được kích hoạt, Kernel đóng gói thông tin của gói tin và gửi đến User Space thông qua Netlink Socket. Netlink là cơ chế giao tiếp giữa Linux Kernel và các tiến trình trong không gian người dùng. Việc sử dụng Netlink giúp giảm sự phụ thuộc vào Kernel Log và tăng tính linh hoạt trong quá trình xử lý dữ liệu.

Bước 3. Input Plugin tiếp nhận dữ liệu

Plugin NFLOG trong ulogd2 nhận dữ liệu từ Netlink và chuyển dữ liệu vào Pipeline. Tại thời điểm này dữ liệu vẫn còn ở dạng thô (Raw Packet).

Bước 4. Filter Plugin xử lý dữ liệu

Các Filter Plugin lần lượt xử lý dữ liệu:

- Phân tích Header.
- Chuyển đổi địa chỉ IP.
- Xác định Interface.
- Trích xuất giao thức.
- Chuyển đổi dữ liệu sang dạng dễ đọc.
- Bổ sung thông tin cần thiết.

Sau bước này, dữ liệu đã ở dạng có thể ghi nhận hoặc xuất ra ngoài.

Bước 5. Output Plugin ghi dữ liệu

Cuối cùng, Output Plugin sẽ ghi dữ liệu theo định dạng mong muốn. Trong phạm vi báo cáo:

- LOGEMU ghi dữ liệu dạng văn bản vào tệp full.log.
- JSON ghi dữ liệu dạng JSON vào tệp firewall_log.json.

Nhờ đó, cùng một luồng dữ liệu đầu vào có thể được xuất đồng thời dưới nhiều định dạng khác nhau, đáp ứng cả nhu cầu kiểm tra trực tiếp và tích hợp với các hệ thống phân tích nhật ký.

3 NGHIÊN CỨU CÁC INPUT PLUGINS (ULOG, NFLOG, NFCT)

Input Plugin là thành phần đầu tiên trong kiến trúc Pipeline của ulogd2, có nhiệm vụ thu thập dữ liệu từ Linux Kernel và chuyển vào chuỗi xử lý của ulogd2. Dữ liệu đầu vào có thể là từng gói tin (Packet) hoặc thông tin về trạng thái kết nối (Connection Tracking). ulogd2 hiện hỗ trợ nhiều Input Plugin, trong đó phổ biến nhất là ULOG, NFLOG và NFCT. Mỗi plugin được thiết kế nhằm phục vụ những mục đích khác nhau và có cơ chế hoạt động riêng.

3.1 Input Plugin ULOG

3.1.1 Giới thiệu

ULOG (Userspace Logging) là cơ chế ghi log được Netfilter cung cấp từ những phiên bản Linux Kernel đầu tiên nhằm chuyển dữ liệu của các gói tin từ Kernel sang không gian người dùng (User Space). Khác với target LOG, vốn chỉ ghi thông tin vào Kernel Log, ULOG sử dụng giao thức Netlink để truyền dữ liệu tới các chương trình xử lý log như ulogd.

Plugin ULOG trong ulogd2 có nhiệm vụ tiếp nhận các gói tin được gửi bởi target ULOG của iptables, sau đó chuyển dữ liệu vào Pipeline để tiếp tục xử lý. Tuy nhiên, cùng với sự phát triển của Netfilter, ULOG đã bộc lộ nhiều hạn chế về hiệu năng và khả năng mở rộng. Do đó, plugin này hiện được xem là cơ chế cũ (legacy) và hầu như đã được thay thế bởi NFLOG trong các hệ thống Linux hiện đại.

3.1.2 Ưu điểm

- Không làm tăng dung lượng Kernel Log.
- Tách biệt quá trình ghi log khỏi Kernel.
- Có thể ghi nhận đầy đủ nội dung gói tin.
- Hỗ trợ xử lý dữ liệu trong User Space.

3.1.3 Nhược điểm

- Chỉ hỗ trợ trên các phiên bản Kernel cũ.
- Hiệu năng không cao khi lưu lượng mạng lớn.
- Không còn được phát triển và duy trì.
- Đã bị thay thế bởi NFLOG trong Netfilter hiện đại.
- Không được khuyến nghị sử dụng trên các hệ thống Linux mới.

3.1.4 Trường hợp sử dụng

Hiện nay ULOG chủ yếu được sử dụng trong:

- Các hệ thống Linux cũ.
- Hệ thống kế thừa (Legacy System).
- Các môi trường cần tương thích với cấu hình iptables cũ.

Đối với các hệ thống mới như WebOS, OpenWrt hoặc các bản phân phối Linux hiện đại, ULOG hầu

như không còn được sử dụng.

3.2 Input Plugin NFLOG

3.2.1 Giới thiệu

NFLOG (Netfilter Logging) là cơ chế ghi log hiện đại của Netfilter, được phát triển để thay thế ULOG. Đây cũng là Input Plugin được sử dụng phổ biến nhất trong ulogd2 và là plugin được triển khai trong báo cáo này.

Khác với target LOG, NFLOG không ghi dữ liệu trực tiếp vào Kernel Log mà chuyển thông tin của gói tin sang User Space thông qua Netlink Socket. Điều này giúp giảm tải cho Kernel, đồng thời cho phép xử lý dữ liệu linh hoạt bằng các Plugin của ulogd2. NFLOG hỗ trợ phân nhóm (Group) các gói tin, cho phép nhiều tiến trình hoặc nhiều Pipeline khác nhau xử lý các loại lưu lượng riêng biệt.

3.2.2 Quy trình hoạt động

- Một gói tin đi qua Netfilter và khớp với Rule sử dụng target NFLOG.
- Kernel sao chép thông tin của gói tin vào NFLOG Queue.
- Dữ liệu được gửi qua Netlink Socket đến User Space.
- Plugin libulogd_inppkt_NFLOG.so tiếp nhận dữ liệu.
- Dữ liệu được chuyển qua các Filter Plugin.
- Output Plugin ghi dữ liệu ra file hoặc cơ sở dữ liệu.

3.2.3 Các tham số của NFLOG

Target NFLOG hỗ trợ nhiều tham số để điều khiển quá trình ghi log.

a. –nflog-group

Xác định nhóm (Group ID) mà gói tin sẽ được gửi tới. Các nhóm giúp phân loại lưu lượng và cho phép nhiều Pipeline khác nhau xử lý các loại gói tin riêng biệt. Ví dụ: –nflog-group 169 — gói tin sẽ được gửi tới Group 169.

b. –nflog-prefix

Thêm chuỗi ký tự vào đầu mỗi bản ghi log nhằm giúp người quản trị dễ dàng nhận biết nguồn gốc của log. Ví dụ: `--nflog-prefix "INPUT"`. Nhờ đó có thể phân biệt log của từng Chain hoặc từng chính sách Firewall.

c. –nflog-size

Xác định số byte tối đa của gói tin sẽ được sao chép sang User Space. Ví dụ: –nflog-size 256 — chỉ 256 byte đầu tiên của gói tin được chuyển tới ulogd2. Việc giới hạn kích thước giúp giảm bộ nhớ và tăng hiệu năng khi xử lý lưu lượng lớn.

d. –nflog-threshold

Quy định số lượng gói tin cần tích lũy trước khi Kernel gửi một lần tới User Space. Ví dụ: –nflog-

threshold 10 — Kernel sẽ gom 10 gói tin rồi mới gửi sang ulogd2. Giá trị này giúp giảm số lần giao tiếp giữa Kernel và User Space, từ đó cải thiện hiệu năng của hệ thống.

3.2.4 Ví dụ cấu hình NFLOG

Ví dụ dưới đây cấu hình iptables để ghi log các gói tin đi vào hệ thống:

```
iptables -A INPUT \
    -j NFLOG \
    --nflog-group 169 \
    --nflog-prefix "INPUT"
```

Giải thích:

- -A INPUT: thêm một Rule vào Chain INPUT, áp dụng cho các gói tin đi vào hệ thống.
- -j NFLOG: khi Rule khớp, gói tin sẽ được chuyển đến target NFLOG thay vì ghi trực tiếp vào Kernel Log.
- –nflog-group 169: đưa gói tin vào Group 169, để ulogd2 có thể lắng nghe và xử lý đúng nhóm lưu lượng.
- `--nflog-prefix "INPUT"`: thêm chuỗi INPUT vào đầu bản ghi log, giúp phân biệt với các bản ghi từ những Chain khác.

3.2.5 Ưu điểm

- Được hỗ trợ trên các phiên bản Linux Kernel hiện đại.
- Hiệu năng cao hơn ULOG.
- Hỗ trợ phân nhóm lưu lượng thông qua Group ID.
- Có thể kết hợp linh hoạt với ulogd2 để xuất nhiều định dạng log khác nhau.
- Giảm tải cho Kernel Log.
- Dễ tích hợp với các hệ thống giám sát và phân tích an toàn thông tin.

3.2.6 Nhược điểm

- Cần cài đặt và cấu hình thêm ulogd2 để xử lý dữ liệu.
- Quá trình cấu hình phức tạp hơn target LOG.
- Nếu cấu hình Group không chính xác, dữ liệu sẽ không được thu thập.
- Khi ghi nhận quá nhiều lưu lượng mà không giới hạn bằng –limit hoặc –nflog-threshold, hiệu năng của hệ thống có thể bị ảnh hưởng.

3.2.7 Trường hợp sử dụng

NFLOG được sử dụng rộng rãi trong:

- Hệ thống Firewall Linux, Router và Gateway.
- Thiết bị IoT và hệ thống nhúng.

- WebOS Automotive.
- Các hệ thống giám sát an toàn thông tin cần xuất log dưới dạng JSON hoặc tích hợp với SIEM.

Trong phạm vi báo cáo này, NFLOG được lựa chọn làm Input Plugin chính để thu thập các gói tin từ Netfilter và chuyển tới ulogd2, sau đó kết hợp với LOGEMU và JSON Output Plugin để tạo các tệp full.log và firewall_log.json phục vụ kiểm tra và phân tích.

3.3 Input Plugin NFCT

3.3.1 Giới thiệu

NFCT (Netfilter Connection Tracking) là Input Plugin của ulogd2 dùng để thu thập thông tin từ Connection Tracking Subsystem (conntrack) của Netfilter. Khác với ULOG và NFLOG chỉ ghi nhận từng gói tin (Packet Logging), NFCT tập trung theo dõi trạng thái của các kết nối mạng (Flow Logging).

Trong Linux Kernel, Connection Tracking có nhiệm vụ theo dõi vòng đời của mỗi kết nối mạng. Khi một kết nối mới được thiết lập, thay đổi trạng thái hoặc kết thúc, thông tin sẽ được lưu trong bảng conntrack của Kernel. Plugin NFCT có nhiệm vụ đọc các sự kiện này và chuyển vào Pipeline của ulogd2 để tiếp tục xử lý và ghi log. Nhờ cơ chế này, NFCT đặc biệt phù hợp với các hệ thống cần giám sát phiên kết nối thay vì từng gói tin riêng lẻ, giúp giảm số lượng bản ghi và cung cấp cái nhìn tổng quan về hoạt động của mạng.

3.3.2 Kiến trúc hoạt động

Quy trình hoạt động gồm các bước:

- Một kết nối mới được tạo hoặc thay đổi trạng thái.
- Connection Tracking của Linux Kernel ghi nhận sự kiện.
- Thông tin được gửi tới User Space thông qua Netlink.
- Plugin libulogd_inpflow_NFCT.so tiếp nhận dữ liệu.
- Dữ liệu được chuyển qua các Filter Plugin.
- Output Plugin ghi thông tin ra file hoặc cơ sở dữ liệu.

Khác với NFLOG, dữ liệu mà NFCT xử lý không phải là từng gói tin mà là thông tin về toàn bộ phiên kết nối.

3.3.3 Các trạng thái của Connection Tracking

Trong quá trình theo dõi một phiên kết nối, Connection Tracking sử dụng nhiều trạng thái khác nhau. Một số trạng thái phổ biến gồm:

a. NEW

Đây là trạng thái của một kết nối mới được tạo. Ví dụ: Client gửi gói TCP SYN đầu tiên; một yêu cầu HTTP mới; một truy vấn DNS mới. Khi một kết nối xuất hiện lần đầu tiên, Kernel sẽ sinh sự kiện NEW.

b. ESTABLISHED

Đây là trạng thái của các kết nối đã được thiết lập thành công. Ví dụ: phiên SSH đang hoạt động; kết nối HTTP sau khi hoàn tất bắt tay TCP; phiên HTTPS đang truyền dữ liệu. Đây là trạng thái chiếm phần lớn thời gian trong vòng đời của một kết nối.

c. RELATED

Đây là trạng thái dành cho các kết nối được sinh ra từ một kết nối đã tồn tại. Ví dụ: FTP Data Connection được tạo từ FTP Control Connection; ICMP báo lỗi liên quan đến một phiên TCP đang hoạt động. Các kết nối RELATED thường được Firewall cho phép dựa trên chính sách của kết nối gốc.

d. DESTROY

Đây là trạng thái khi một kết nối kết thúc và bị xóa khỏi bảng Connection Tracking. Nguyên nhân có thể bao gồm: hai phía đóng kết nối TCP; hết thời gian timeout; kết nối bị hủy; Firewall xóa trạng thái. Sự kiện DESTROY thường được sử dụng để thống kê thời gian tồn tại của một phiên kết nối hoặc ghi nhận việc kết thúc phiên giao tiếp.

3.3.4 Ưu điểm

- Ghi nhận theo phiên kết nối thay vì từng gói tin.
- Số lượng bản ghi ít hơn so với Packet Logging.
- Giảm tải cho hệ thống khi lưu lượng lớn.
- Phù hợp với việc thống kê và phân tích lưu lượng.
- Có thể theo dõi vòng đời đầy đủ của một kết nối.

3.3.5 Nhược điểm

- Không lưu toàn bộ nội dung của từng gói tin.
- Không phù hợp cho việc phân tích chi tiết Packet.
- Phụ thuộc vào Connection Tracking của Kernel.
- Một số giao thức đặc biệt có thể không được theo dõi đầy đủ.

3.3.6 Trường hợp sử dụng

NFCT thường được sử dụng trong các hệ thống:

- Thống kê lưu lượng mạng và phân tích Flow.
- Hệ thống giám sát Firewall và Network Accounting.
- Phân tích hành vi kết nối và thu thập dữ liệu cho hệ thống SIEM.

Trong các hệ thống cần điều tra chi tiết từng gói tin, NFLOG vẫn là lựa chọn phù hợp hơn.

3.4 So sánh các Input Plugin

Bảng dưới đây so sánh ba Input Plugin phổ biến của ulogd2.

Tiêu chí	ULOG	NFLOG	NFCT
Loại dữ liệu	Packet	Packet	Flow (Connection)
Ghi từng gói tin	✓	✓	✗
Ghi theo kết nối	✗	✗	✓
Cơ chế hoạt động	Netlink (Legacy)	Netlink	Conntrack + Netlink
Hỗ trợ Linux hiện đại	Hạn chế	✓	✓
Hiệu năng	Trung bình	Cao	Rất cao
Khả năng mở rộng	Thấp	Cao	Cao
Mức độ sử dụng hiện nay	Rất ít	Rất phổ biến	Phổ biến
Trường hợp sử dụng	Hệ thống cũ	Firewall, IDS, Logging	Giám sát Flow, Thống kê

Nhận xét

Qua quá trình nghiên cứu có thể thấy ba Input Plugin đều có nhiệm vụ thu thập dữ liệu từ Linux Kernel, tuy nhiên mục đích sử dụng của chúng khác nhau. ULOG là cơ chế ghi log đời đầu của Netfilter; hiện đã lỗi thời và hầu như không còn được triển khai trên các hệ thống Linux hiện đại do bị thay thế bởi NFLOG. NFLOG là Input Plugin được khuyến nghị sử dụng trong hầu hết các hệ thống hiện nay; đây cũng là plugin được lựa chọn để triển khai trong báo cáo này. NFCT tập trung theo dõi trạng thái các phiên kết nối thay vì từng gói tin, phù hợp với thống kê lưu lượng và giám sát ở mức Flow.

Tùy theo yêu cầu của hệ thống, người quản trị có thể lựa chọn NFLOG khi cần phân tích chi tiết từng gói tin hoặc NFCT khi mục tiêu là theo dõi và thống kê toàn bộ vòng đời của các kết nối mạng. Trong phạm vi báo cáo này, NFLOG được sử dụng làm Input Plugin chính vì đáp ứng tốt yêu cầu ghi nhận các gói tin do iptables sinh ra và hỗ trợ xuất dữ liệu dưới định dạng JSON thông qua ulogd2.

4 NGHIÊN CỨU CÁC OUTPUT PLUGINS

4.1 Tổng quan về Output Plugin

Trong kiến trúc Pipeline của ulogd2, Output Plugin là thành phần cuối cùng chịu trách nhiệm ghi dữ liệu sau khi đã được Input Plugin thu thập và Filter Plugin xử lý. Tùy thuộc vào mục đích sử dụng, Output Plugin có thể lưu dữ liệu dưới dạng văn bản, cơ sở dữ liệu, tệp tin hoặc truyền tới các hệ thống giám sát bên ngoài. Một trong những ưu điểm nổi bật của ulogd2 là khả năng hỗ trợ nhiều Output Plugin khác nhau. Trong phạm vi nghiên cứu này, các Output Plugin được khảo sát bao gồm: LOGEMU, OPRINT, SYSLOG, MYSQL, PGSQL, SQLITE3, PCAP, IPFIX, NACCT và JSON.

4.2 Output Plugin LOGEMU

4.2.1 Giới thiệu

LOGEMU (Log Emulator) là Output Plugin dùng để mô phỏng định dạng log truyền thống của iptables. Plugin này xuất dữ liệu dưới dạng văn bản (Plain Text), giúp người quản trị dễ dàng theo dõi các sự kiện mạng mà không cần sử dụng các công cụ phân tích chuyên dụng.

4.2.2 Nguyên lý hoạt động

Sau khi dữ liệu được xử lý bởi các Filter Plugin, LOGEMU tiếp nhận các trường thông tin như thời gian, địa chỉ IP nguồn, địa chỉ IP đích, giao thức, cổng và giao diện mạng. Các thông tin này được định dạng thành chuỗi văn bản theo phong cách tương tự log của iptables rồi ghi vào file hoặc thiết bị đầu ra được chỉ định trong file cấu hình của ulogd2.

4.2.3 Ưu điểm

- Dễ đọc và dễ kiểm tra.
- Tương thích với định dạng log truyền thống.
- Không yêu cầu phần mềm phân tích chuyên dụng.

4.2.4 Nhược điểm

- Dữ liệu không có cấu trúc.
- Khó xử lý tự động.
- Không phù hợp với hệ thống giám sát quy mô lớn.

4.2.5 Trường hợp sử dụng

LOGEMU thường được sử dụng để kiểm tra nhanh hoạt động của Firewall, theo dõi quá trình thử nghiệm hoặc phục vụ công tác gỡ lỗi.

4.3 Output Plugin OPRINT

4.3.1 Giới thiệu

OPRINT (Output Print) là Output Plugin cho phép xuất dữ liệu theo định dạng văn bản do người dùng tự định nghĩa.

4.3.2 Nguyên lý hoạt động

Plugin tiếp nhận dữ liệu sau khi được xử lý và lựa chọn các trường thông tin theo cấu hình của người dùng. Nội dung đầu ra được tạo theo mẫu định dạng (template) đã khai báo, nhờ đó người quản trị có thể quyết định những trường dữ liệu nào sẽ được hiển thị.

4.3.3 Ưu điểm

- Linh hoạt trong việc định dạng dữ liệu.
- Có thể tùy chỉnh nội dung đầu ra.

4.3.4 Nhược điểm

- Không có định dạng chuẩn.
- Khó tích hợp với các hệ thống phân tích tự động.

4.3.5 Trường hợp sử dụng

Thích hợp cho việc kiểm thử, gỡ lỗi hoặc xây dựng các báo cáo văn bản đơn giản.

4.4 Output Plugin SYSLOG

4.4.1 Giới thiệu

SYSLOG là Output Plugin cho phép ulogd2 chuyển dữ liệu tới dịch vụ Syslog của hệ điều hành Linux.

4.4.2 Nguyên lý hoạt động

Sau khi nhận dữ liệu từ Filter Plugin, SYSLOG sử dụng giao diện Syslog chuẩn của hệ điều hành để gửi bản ghi tới các dịch vụ như rsyslog hoặc syslog-ng. Từ đó dữ liệu được lưu vào các tệp nhật ký của hệ thống hoặc chuyển tiếp tới máy chủ Syslog từ xa nếu được cấu hình.

4.4.3 Ưu điểm

- Tích hợp tốt với hệ điều hành Linux.
- Dễ quản lý tập trung.

4.4.4 Nhược điểm

- Phụ thuộc vào dịch vụ Syslog.
- Dữ liệu không có cấu trúc.

4.4.5 Trường hợp sử dụng

Được sử dụng phổ biến trên các máy chủ Linux, router và gateway.

4.5 Output Plugin MYSQL

4.5.1 Giới thiệu

MYSQL Output Plugin cho phép lưu dữ liệu trực tiếp vào cơ sở dữ liệu MySQL thông qua thư viện DBI.

4.5.2 Nguyên lý hoạt động

Plugin thiết lập kết nối tới máy chủ MySQL và chuyển mỗi bản ghi thành một bản ghi trong bảng dữ liệu tương ứng. Việc lưu trữ theo dạng quan hệ giúp người quản trị dễ dàng truy vấn, thống kê và phân tích thông tin.

4.5.3 Ưu điểm

- Dễ truy vấn dữ liệu.
- Phù hợp cho thống kê và báo cáo.

4.5.4 Nhược điểm

- Cần triển khai máy chủ MySQL.
- Hiệu năng có thể giảm nếu số lượng log quá lớn.

4.5.5 Trường hợp sử dụng

Thích hợp cho các hệ thống quản lý tập trung hoặc các trung tâm giám sát mạng.

4.6 Output Plugin PGSQL

4.6.1 Giới thiệu

PGSQL Output Plugin có chức năng tương tự MYSQL nhưng sử dụng hệ quản trị cơ sở dữ liệu PostgreSQL.

4.6.2 Nguyên lý hoạt động

Plugin kết nối tới PostgreSQL và lưu mỗi bản ghi của ulogd2 thành một bản ghi trong cơ sở dữ liệu, hỗ trợ truy vấn và quản lý dữ liệu lâu dài.

4.6.3 Ưu điểm

- Độ ổn định cao.
- Quản lý dữ liệu hiệu quả.

4.6.4 Nhược điểm

- Cần triển khai PostgreSQL.
- Cấu hình phức tạp hơn so với lưu file.

4.6.5 Trường hợp sử dụng

Thường được sử dụng trong các hệ thống doanh nghiệp hoặc trung tâm dữ liệu.

4.7 Output Plugin SQLITE3

4.7.1 Giới thiệu

SQLITE3 Output Plugin lưu dữ liệu vào cơ sở dữ liệu SQLite cục bộ.

4.7.2 Nguyên lý hoạt động

Plugin sử dụng thư viện SQLite để tạo hoặc mở tệp cơ sở dữ liệu, sau đó ghi các bản ghi của ulogd2 vào các bảng tương ứng. Do SQLite hoạt động dưới dạng một tệp duy nhất nên việc triển khai khá đơn giản.

4.7.3 Ưu điểm

- Không cần máy chủ cơ sở dữ liệu.
- Dễ triển khai.
- Phù hợp với thiết bị nhúng.

4.7.4 Nhược điểm

- Không thích hợp với hệ thống có lượng truy cập lớn.
- Khả năng mở rộng hạn chế.

4.7.5 Trường hợp sử dụng

Thích hợp với Raspberry Pi, thiết bị IoT và các hệ thống nhúng.

4.8 Output Plugin PCAP

4.8.1 Giới thiệu

PCAP Output Plugin ghi dữ liệu theo định dạng `.pcap`, cho phép mở và phân tích bằng các phần mềm như Wireshark hoặc tcpdump.

4.8.2 Nguyên lý hoạt động

Plugin ghi trực tiếp dữ liệu gói tin vào tệp PCAP theo đúng định dạng chuẩn của libpcap. Điều này giúp giữ nguyên thông tin cần thiết để phân tích chi tiết lưu lượng mạng.

4.8.3 Ưu điểm

- Hỗ trợ phân tích chuyên sâu.
- Tương thích với nhiều công cụ mạng.

4.8.4 Nhược điểm

- Dung lượng lưu trữ lớn.
- Không phù hợp để ghi log lâu dài.

4.8.5 Trường hợp sử dụng

Phân tích sự cố, điều tra tấn công hoặc nghiên cứu lưu lượng mạng.

4.9 Output Plugin IPFIX

4.9.1 Giới thiệu

IPFIX (IP Flow Information Export) là Output Plugin xuất thông tin lưu lượng theo chuẩn IPFIX do IETF ban hành.

4.9.2 Nguyên lý hoạt động

Plugin tổng hợp thông tin của các phiên kết nối và xuất dữ liệu theo định dạng IPFIX để gửi tới các hệ thống thu thập và phân tích Flow.

4.9.3 Ưu điểm

- Tuân theo chuẩn quốc tế.
- Phù hợp cho giám sát lưu lượng.

4.9.4 Nhược điểm

- Không lưu chi tiết từng gói tin.
- Cần hệ thống Collector hỗ trợ IPFIX.

4.9.5 Trường hợp sử dụng

Được sử dụng trong các hệ thống giám sát mạng, ISP và trung tâm dữ liệu.

4.10 Output Plugin NACCT

4.10.1 Giới thiệu

NACCT (Network Accounting) là Output Plugin phục vụ mục đích thống kê và kế toán lưu lượng mạng.

4.10.2 Nguyên lý hoạt động

Plugin ghi nhận các thông tin như số lượng gói tin, số byte truyền nhận và các thông tin thống kê khác của từng kết nối, phục vụ việc tính toán lưu lượng sử dụng.

4.10.3 Ưu điểm

- Hỗ trợ thống kê lưu lượng.
- Phù hợp cho các hệ thống tính cước.

4.10.4 Nhược điểm

- Không ghi chi tiết nội dung gói tin.
- Chỉ phù hợp với các bài toán thống kê.

4.10.5 Trường hợp sử dụng

Được sử dụng trong các hệ thống quản lý băng thông, ISP hoặc Network Accounting.

4.11 Output Plugin JSON

4.11.1 Giới thiệu

JSON (JavaScript Object Notation) là Output Plugin cho phép ulogd2 xuất dữ liệu theo định dạng JSON. Đây là một trong những plugin được sử dụng phổ biến nhất hiện nay nhờ khả năng lưu trữ dữ liệu có cấu trúc và dễ dàng tích hợp với các hệ thống phân tích nhật ký.

4.11.2 Nguyên lý hoạt động

Sau khi dữ liệu được xử lý bởi các Filter Plugin, JSON Plugin chuyển đổi từng trường thông tin thành các cặp khóa – giá trị (key–value) theo chuẩn JSON. Plugin sử dụng thư viện libjansson để tạo và ghi các đối tượng JSON vào tệp đầu ra. Mỗi bản ghi được lưu thành một đối tượng JSON độc lập, giúp các hệ thống phân tích hoặc các ứng dụng xử lý dữ liệu có thể dễ dàng đọc và khai thác.

4.11.3 Ưu điểm

- Dữ liệu có cấu trúc rõ ràng.
- Dễ dàng phân tích và xử lý tự động.
- Tương thích với các hệ thống ELK Stack, Splunk, Wazuh và SIEM.

4.11.4 Nhược điểm

- Phụ thuộc vào thư viện libjansson.
- Kích thước tệp có thể tăng nhanh khi ghi nhận lượng lớn dữ liệu.

4.11.5 Trường hợp sử dụng

JSON là định dạng được sử dụng rộng rãi trong các hệ thống SOC, SIEM, phân tích nhật ký tập trung và các ứng dụng trí tuệ nhân tạo. Trong phạm vi báo cáo này, JSON được lựa chọn làm định dạng đầu ra chính theo yêu cầu của doanh nghiệp giả định VanFit, đồng thời kết hợp với LOGEMU để tạo hai tệp firewall_log.json và full.log phục vụ quá trình kiểm thử và đánh giá hệ thống.

4.12 So sánh các Output Plugin

Plugin	Định dạng đầu ra	Có cấu trúc	Khả năng phân tích tự động	Mục đích sử dụng
LOGEMU	Văn bản	✗	Thấp	Kiểm tra, gỡ lỗi
OPRINT	Văn bản tùy chỉnh	✗	Thấp	Báo cáo đơn giản
SYSLOG	Syslog	✗	Trung bình	Nhật ký hệ thống
MYSQL	MySQL	✓	Cao	Lưu trữ tập trung
PGSQL	PostgreSQL	✓	Cao	Hệ thống doanh nghiệp
SQLITE3	SQLite	✓	Cao	Thiết bị nhúng
PCAP	`.pcap`	✓	Cao	Phân tích gói tin
IPFIX	Flow	✓	Cao	Giám sát lưu lượng
NACCT	Accounting	✓	Cao	Thống kê lưu lượng
JSON	JSON	✓	Rất cao	SIEM, AI, SOC

5 KHẢO SÁT ULOGD2 TRÊN WEBOS

5.1 Giới thiệu

Qua kết quả khảo sát ban đầu, hệ thống webOS OSE chưa tích hợp package ulogd2, dẫn đến BitBake không thể nhận diện và biên dịch package này. Đồng thời, trên Raspberry Pi cũng không tồn tại daemon ulogd, các plugin, file cấu hình hay service liên quan. Do đó, trước khi cấu hình Input/Output Plugin, cần bổ sung ulogd2 vào hệ thống build của WebOS. Chương này trình bày quá trình tích hợp recipe ulogd2 từ OpenEmbedded, biên dịch lại image WebOS, sau đó khảo sát các thành phần của ulogd2 trên thiết bị Raspberry Pi.

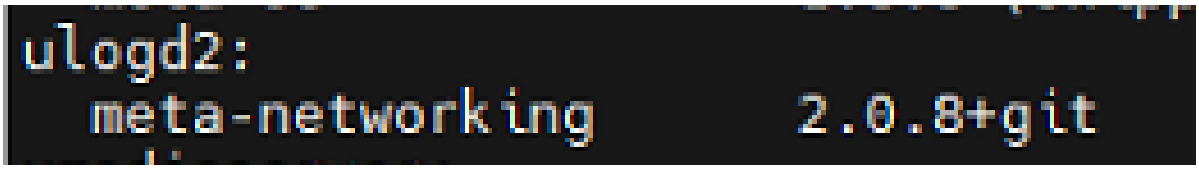
5.2 Tích hợp recipe ulogd2 vào hệ thống build

Source của webOS OSE chưa tích hợp recipe của ulogd2, do đó BitBake không thể nhận diện và biên dịch package này. Để khắc phục, nhóm sử dụng recipe có sẵn từ dự án OpenEmbedded — nguồn recipe chính của nhiều bản phân phối Linux xây dựng trên Yocto Project.

Trước tiên, source của layer meta-openembedded được tải về từ kho lưu trữ chính thức. Sau đó tiến hành tìm kiếm recipe của ulogd2 trong layer meta-networking và lựa chọn phiên bản phù hợp với môi trường build của webOS OSE. Recipe này được sao chép vào layer meta-oe/meta-networking/recipes-filter của dự án WebOS để BitBake có thể nhận diện trong quá trình biên dịch.

Quy trình tích hợp được thực hiện theo sơ đồ sau:

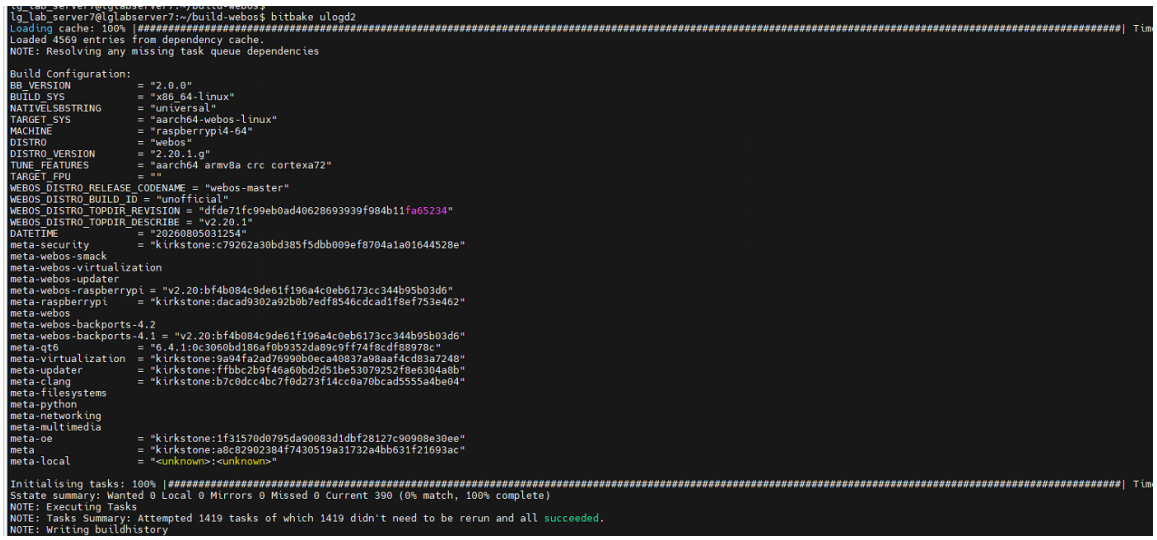
```
Clone meta-openembedded
          |
          ▼
Tìm recipe ulogd2
          |
          ▼
Sao chép recipe vào
meta-oe/meta-networking/recipes-filter
          |
          ▼
BitBake nhận diện recipe
          |
          ▼
Biên dịch package ulogd2
          |
          ▼
Build lại webos-image
```



Hình 5.1: Minh họa quy trình tích hợp (phần liên quan)

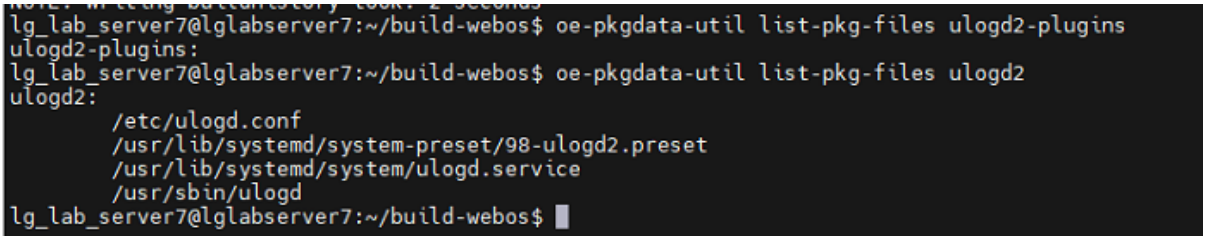
Sau khi bổ sung recipe, tiến hành sử dụng lệnh bitbake-layers show-recipes để kiểm tra BitBake

đã nhận diện package ulogd2. Tiếp theo, package được biên dịch độc lập nhằm kiểm tra khả năng build thành công trước khi tích hợp vào toàn bộ image của WebOS. Cuối cùng, thực hiện build lại webos-image để đưa package ulogd2 vào Root File System của hệ điều hành.



Hình 5.2: Kết quả build / nhận diện recipe ulogd2

5.3 Kiểm tra kết quả build



Hình 5.3: Kiểm tra package sau khi build

Sau khi quá trình build hoàn tất, sử dụng công cụ oe-pkgdata-util để kiểm tra package được sinh ra từ BitBake và xác nhận các tệp đã được đóng gói thành công. Kết quả cho thấy package ulogd2 cùng các plugin liên quan đã được tạo thành công và sẵn sàng tích hợp vào image của WebOS. Điều này chứng minh recipe đã được bổ sung đúng cách và hệ thống build có thể biên dịch ulogd2 mà không cần xây dựng recipe mới.

Sau khi image mới được nạp lên Raspberry Pi, hệ điều hành đã có đầy đủ daemon ulogd2 và các thành phần cần thiết để tiếp tục cấu hình cơ chế ghi log sử dụng NFLOG.

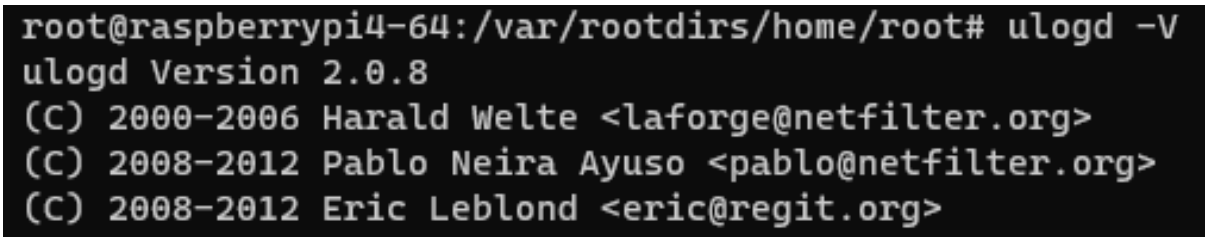
5.4 Kiểm tra phiên bản ulogd2 trên thiết bị

Để xác định ulogd2 đã được cài đặt thành công, sử dụng lệnh:

```
ulogd -V
```

Lệnh trên hiển thị phiên bản của ulogd2 đang được cài đặt trên hệ thống. Việc kiểm tra phiên bản

giúp xác nhận daemon đã được tích hợp thành công vào image WebOS và có thể sử dụng cho các bước cấu hình tiếp theo.



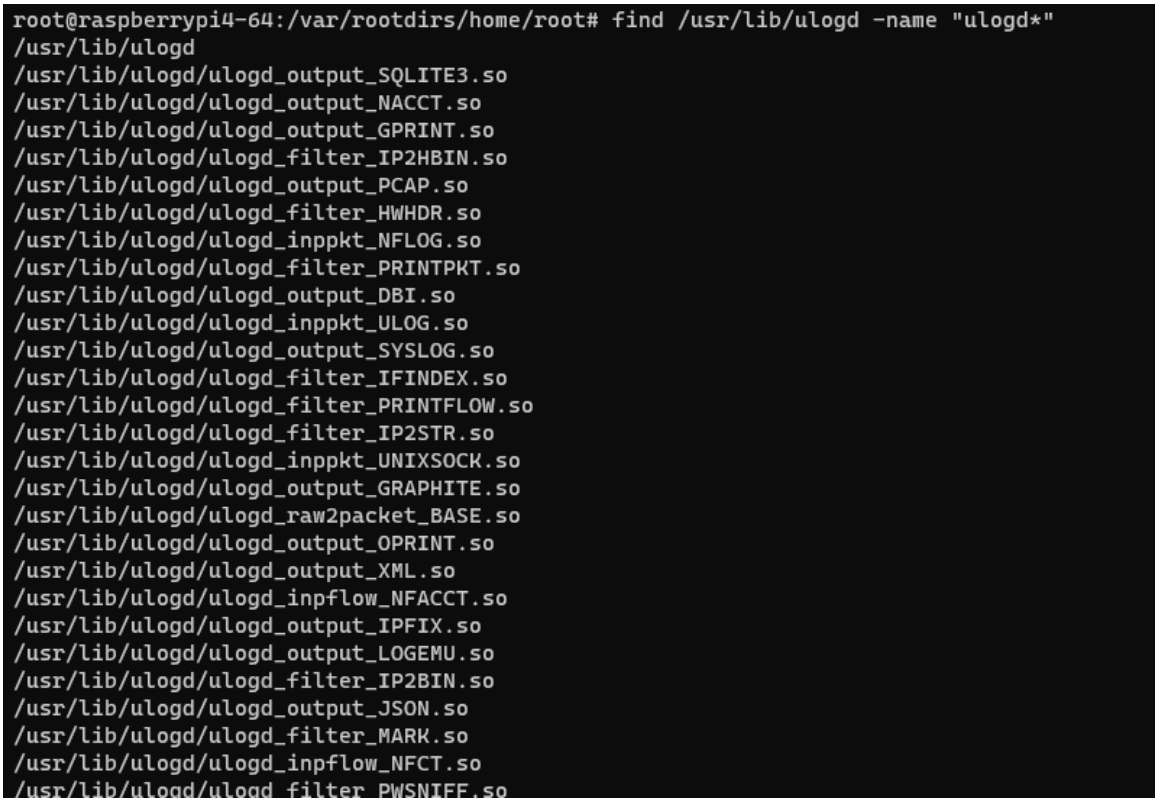
Hình 5.4: Kết quả lệnh ulogd -V

5.5 Kiểm tra các Plugin của ulogd2

Sau khi xác nhận daemon hoạt động, tiến hành kiểm tra các thư viện Plugin được cài đặt bằng lệnh:

```
find /usr/lib/ulogd -name "ulogd*"
```

Trong thư mục này chứa các thư viện động (.so) tương ứng với các Input Plugin, Filter Plugin và Output Plugin của ulogd2.



Hình 5.5: Danh sách plugin trong /usr/lib/ulogd

5.6 Danh sách Plugin được hỗ trợ

Dựa trên kết quả khảo sát, các plugin của ulogd2 được phân loại như sau.

5.6.1 Input Plugin

Plugin	Thư viện
NFLOG	ulogd_inppkt_NFLOG.so
ULOG	ulogd_inppkt_ULOG.so
NFCT	ulogd_inpflow_NFCT.so
NFACCT	ulogd_inpflow_NFACCT.so
UNIXSOCK	ulogd_inppkt_UNIXSOCK.so

Các plugin trên cho phép ulogd2 thu thập dữ liệu từ nhiều nguồn khác nhau như Netfilter, Connection Tracking hoặc Unix Socket.

5.6.2 Filter Plugin

Plugin	Thư viện
BASE	ulogd_raw2packet_BASE.so
IFINDEX	ulogd_filter_IFINDEX.so
IP2STR	ulogd_filter_IP2STR.so
IP2BIN	ulogd_filter_IP2BIN.so
IP2HBIN	ulogd_filter_IP2HBIN.so
HWHDR	ulogd_filter_HWHDR.so
MARK	ulogd_filter_MARK.so
PRINTPKT	ulogd_filter_PRINTPKT.so
PRINTFLOW	ulogd_filter_PRINTFLOW.so
PWSNIFF	ulogd_filter_PWSNIFF.so

Các Filter Plugin có nhiệm vụ chuyển đổi và bổ sung thông tin trước khi dữ liệu được ghi ra ngoài.

5.6.3 Output Plugin

Plugin	Thư viện
LOGEMU	ulogd_output_LOGEMU.so
OPRINT	ulogd_output_OPRINT.so
SYSLOG	ulogd_output_SYSLOG.so
JSON	ulogd_output_JSON.so
XML	ulogd_output_XML.so
SQLITE3	ulogd_output_SQLITE3.so
DBI	ulogd_output_DBI.so
PCAP	ulogd_output_PCAP.so
IPFIX	ulogd_output_IPFIX.so
NACCT	ulogd_output_NACCT.so
GRAPHITE	ulogd_output_GRAPHITE.so
GPRINT	ulogd_output_GPRINT.so

Các Output Plugin cho phép xuất dữ liệu theo nhiều định dạng hoặc lưu trữ trên các hệ thống khác nhau.

5.7 Đánh giá khả năng hỗ trợ

Từ kết quả khảo sát có thể nhận thấy hệ thống WebOS sau khi tích hợp đã hỗ trợ đầy đủ các Input Plugin và Output Plugin cần thiết cho quá trình triển khai. Bảng dưới đây tổng hợp các plugin chính được sử dụng trong bài thực hành.

Plugin	Hỗ trợ	Vai trò
ULOG	✓	Ghi log Packet (Legacy)
NFLOG	✓	Ghi log Packet từ Netfilter
NFCT	✓	Ghi log Connection Tracking
LOGEMU	✓	Ghi log dạng văn bản
OPRINT	✓	Xuất dữ liệu dạng Text
SYSLOG	✓	Gửi log tới Syslog
JSON	✓	Xuất log định dạng JSON
SQLITE3	✓	Lưu vào SQLite
DBI	✓	Ghi vào cơ sở dữ liệu
PCAP	✓	Xuất dữ liệu dạng PCAP
IPFIX	✓	Xuất thông tin Flow

Kết quả này cho thấy hệ thống đã đáp ứng đầy đủ các yêu cầu của bài thực hành, đặc biệt là sự xuất hiện của Input Plugin NFLOG, Input Plugin NFCT, Output Plugin LOGEMU và Output Plugin JSON. Đây là các thành phần sẽ được sử dụng trong chương tiếp theo để xây dựng pipeline ghi log, cấu hình ulogd.conf, kết hợp với các luật iptables và xuất dữ liệu ra hai tệp full.log và firewall_log.json.

6 XÂY DỰNG ULOGD TÙY BIẾN VÀ TRIỂN KHAI THỰC TẾ

Sau khi hoàn tất tích hợp và khảo sát plugin trên WebOS, chương này trình bày việc xây dựng cấu hình ulogd2 tùy biến theo yêu cầu thực tế: xuất log song song dạng văn bản và JSON, quy hoạch NFLOG group, cấu hình /etc/ulogd.conf hoàn chỉnh và chi tiết các test case kiểm chứng trên Raspberry Pi.

6.1 Yêu cầu của VanFit và hỗ trợ xuất JSON (libjansson)

Theo yêu cầu triển khai của doanh nghiệp giả định VanFit, hệ thống cần ghi nhận nhật ký firewall/Netfilter dưới định dạng có cấu trúc (JSON) để dễ dàng tích hợp với hệ thống phân tích log hoặc các nền tảng AI phục vụ phát hiện bất thường trong tương lai, đồng thời vẫn giữ kênh log văn bản để kiểm tra trực tiếp trên thiết bị.

Từ yêu cầu đó, giải pháp triển khai được xác định như sau:

- Input: NFLOG (packet logging từ iptables/ip6tables).
- Output song song: LOGEMU → /var/log/full.log; JSON → /var/log/firewall_log.json.
- JSON Output Plugin phụ thuộc thư viện libjansson để tạo và ghi các đối tượng JSON; cần đảm bảo thư viện này có trong môi trường build/runtime để plugin JSON hoạt động.
- Điều chỉnh/biên dịch ulogd2 (kèm recipe/package dependency) để hỗ trợ xuất JSON, tạo tiền đề tích hợp SIEM/AI.

Mục tiêu vận hành cụ thể: tích hợp ulogd2 vào hệ thống firewall (iptables/ip6tables) trên Raspberry Pi (WebOS) nhằm ghi lại bằng chứng (audit log) cho các gói tin khớp rule quan trọng — cả traffic bị chặn (tấn công, vi phạm chính sách) lẫn traffic hợp lệ được chấp nhận. Bộ test case trong báo cáo mở rộng và liên kết với các cơ chế đã xây dựng ở tuần 1 (phòng chống DDoS), tuần 2 (VLAN/IPv6) và tuần 3 (Unicast/Multicast/Broadcast/Anycast), nhằm chứng minh mỗi cơ chế phòng thủ đều có bằng chứng log tương ứng.

6.2 Nguyên tắc đặt vị trí rule NFLOG

Các nguyên tắc sau được áp dụng cho toàn bộ test case:

- Muốn log traffic BỊ CHẶN bởi một rule DROP cụ thể → đặt NFLOG ngay TRƯỚC dòng DROP đó, cùng chain/table.
- Muốn log traffic HỢP LỆ được chấp nhận (audit) → đặt NFLOG ở CUỐI chain, sau khi mọi rule lọc đã chạy.
- Mỗi luồng log (nflog-group) chỉ được một tiến trình lắng nghe tại một thời điểm — hai stack chạy trong cùng một tiến trình ulogd2 phải dùng hai group khác nhau nếu muốn tách kênh riêng, hoặc dùng chung group nếu muốn cùng một nguồn traffic xuất ra nhiều định dạng qua nhiều input-instance cùng group.

Trong thực tế triển khai tuần 4, để tránh xung đột bind group (xem Chương 7), mỗi loại traffic dùng hai rule iptables đẩy lên hai group độc lập theo quy ước: Group_JSON = Group_TEXT + 100.

6.3 Bảng tổng hợp NFLOG group

Group	Traffic	Output
10	ICMP (ping) – IPv4	full.log (LOGEMU)
11	ICMP (ping) – IPv4	firewall_log.json (JSON)
12/112	TCP SYN flood (port 80)	full.log + JSON
13/113	Broadcast IPv4 (192.168.1.255)	full.log + JSON
14/114	Multicast IPv4 (224.0.0.0/4)	full.log + JSON
15/115	Smurf attack (ICMP spoof)	full.log + JSON
16/116	OUTPUT audit (traffic hợp lệ)	full.log + JSON

6.4 Cấu hình /etc/ulogd.conf hoàn chỉnh

Phần [global] – nạp plugin:

```
plugin="/usr/lib/ulogd/ulogd_inppkt_NFLOG.so"
plugin="/usr/lib/ulogd/ulogd_raw2packet_BASE.so"
plugin="/usr/lib/ulogd/ulogd_filter_IFINDEX.so"
plugin="/usr/lib/ulogd/ulogd_filter_HWHDR.so"
plugin="/usr/lib/ulogd/ulogd_filter_IP2STR.so"
plugin="/usr/lib/ulogd/ulogd_filter_PRINTPKT.so"
plugin="/usr/lib/ulogd/ulogd_output_LOGEMU.so"
plugin="/usr/lib/ulogd/ulogd_output_JSON.so"
```

Các stack (theo nhóm traffic):

```
#ICMP IPv4
stack=log1:NFLOG,base1:BASE,ifi1:IFINDEX,hw1:HWHDR,ip1:IP2STR,print1:PRINTPKT,emu
↪  1:LOGEMU
stack=log2:NFLOG,base2:BASE,ifi2:IFINDEX,hw2:HWHDR,ip2:IP2STR,json2:JSON

#SYN
stack=log3:NFLOG,base3:BASE,ifi3:IFINDEX,hwhdr3:HWHDR,ip3:IP2STR,print3:PRINTPKT,
↪  emu3:LOGEMU
stack=log3j:NFLOG,base3j:BASE,ifi3j:IFINDEX,hwhdr3j:HWHDR,ip3j:IP2STR,json3j:JSON

#Broadcast IPv4
stack=log4:NFLOG,base4:BASE,ifi4:IFINDEX,hwhdr4:HWHDR,ip4:IP2STR,print4:PRINTPKT,
↪  emu4:LOGEMU
stack=log4j:NFLOG,base4j:BASE,ifi4j:IFINDEX,hwhdr4j:HWHDR,ip4j:IP2STR,json4j:JSON

#Multicast IPv4
stack=log5:NFLOG,base5:BASE,ifi5:IFINDEX,hwhdr5:HWHDR,ip5:IP2STR,print5:PRINTPKT,
↪  emu5:LOGEMU
stack=log5j:NFLOG,base5j:BASE,ifi5j:IFINDEX,hwhdr5j:HWHDR,ip5j:IP2STR,json5j:JSON

#Smurf
```

```
stack=log6:NFLOG,base6:BASE,ifi6:IFINDEX,hwhdr6:HWHDR,ip6:IP2STR,print6:PRINTPKT,
↪  emu6:LOGEMU
stack=log6j:NFLOG,base6j:BASE,ifi6j:IFINDEX,hwhdr6j:HWHDR,ip6j:IP2STR,json6j:JSON

#OUTPUT
stack=log7:NFLOG,base7:BASE,ifi7:IFINDEX,hwhdr7:HWHDR,ip7:IP2STR,print7:PRINTPKT,
↪  emu7:LOGEMU
stack=log7j:NFLOG,base7j:BASE,ifi7j:IFINDEX,hwhdr7j:HWHDR,ip7j:IP2STR,json7j:JSON
```

Ánh xạ group cho các input instance:

```
[log1]
group=10

[log2]
group=11

[log3]
group=12

[log3j]
group=112

[log4]
group=13

[log4j]
group=113

[log5]
group=14

[log5j]
group=114
```

```
[log6]
group=15

[log6j]
group=115

[log7]
group=16

[log7j]
group=116
```

Cấu hình file đầu ra LOGEMU / JSON:

```
[emu1]
file=”/var/log/full.log”
```

```
sync=1

[json2]
sync=1
file="/var/log/firewall_log.json"

[emu3]
file="/var/log/full.log"
sync=1

[json3j]
sync=1
file="/var/log/firewall_log.json"

[emu4]
file="/var/log/full.log"
sync=1

[json4j]
sync=1
file="/var/log/firewall_log.json"
```

```
[emu5]
file="/var/log/full.log"
sync=1

[json5j]
sync=1
file="/var/log/firewall_log.json"

[emu6]
file="/var/log/full.log"
sync=1

[json6j]
sync=1
file="/var/log/firewall_log.json"

[emu7]
file="/var/log/full.log"
sync=1

[json7j]
sync=1
file="/var/log/firewall_log.json"
```

6.5 Chi tiết các Test Case

6.5.1 TC-W4-01 — Log ICMP Ping IPv4

Mục tiêu: Kiểm tra ulogd2 nhận dữ liệu từ NFLOG và xuất đúng ra 2 định dạng full.log / firewall_log.json.

1. Rule iptables

```
$IPT -t mangle -A PREROUTING -p icmp -j NFLOG --nflog-prefix "ICMP_LOG"
↪  --nflog-group 10
$IPT -t mangle -A PREROUTING -p icmp -j NFLOG --nflog-prefix "ICMP_LOG_JSON"
↪  --nflog-group 11
```

2. Stack ulogd2

```
stack=log1:NFLOG,base1:BASE,ifi1:IFINDEX,hwhdr1:HWHDR,ip1:IP2STR,print1:PRINTPKT,
↪  emu1:LOGEMU
stack=log2:NFLOG,base2:BASE,ifi2:IFINDEX,hwhdr2:HWHDR,ip2:IP2STR,json1:JSON
```

```
### mangle/PREROUTING: ACCEPT truoc, ctstate INVALID DROP sau ###
$IPT -t mangle -A PREROUTING -d 224.0.0.0/4 -p udp --dport 5007 -j ACCEPT      # Multicast
$IPT -t mangle -A PREROUTING -d 192.168.1.255 -p udp --dport 5008 -j ACCEPT    # Broadcast
$IPT -t mangle -A PREROUTING -p icmp -j NFLOG --nflog-prefix "ICMP_LOG" --nflog-group 10
$IPT -t mangle -A PREROUTING -p icmp -j NFLOG --nflog-prefix "ICMP_LOG_JSON" --nflog-group 11
$IPT -t mangle -A PREROUTING -m conntrack --ctstate INVALID -j DROP
$IPT -t mangle -A PREROUTING -p tcp ! --syn -m conntrack --ctstate NEW -j DROP
```

3. Các bước thực hiện

Trên Raspberry: mở 2 tcpdump song song để xác nhận kênh có dữ liệu:

```
root@raspberrypi4-64:/etc/firewall# tcpdump -ni nflog:10 -ttt
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on nflog:10, link-type NFLOG (Linux netfilter log messages), snapshot le
ngth 262144 bytes
 00:00:00.000000 IP 192.168.1.144 > 192.168.1.185: ICMP echo request, id 1, seq 40
, length 40
 00:00:01.023968 IP 192.168.1.144 > 192.168.1.185: ICMP echo request, id 1, seq 41
, length 40
 00:00:00.000172 IP 192.168.1.144 > 192.168.1.185: ICMP echo request, id 1, seq 42
, length 40
 00:00:02.015774 IP 192.168.1.144 > 192.168.1.185: ICMP echo request, id 1, seq 43
, length 40
```

```
tcpdump -ni nflog:10 -ttt
```

```
root@raspberrypi4-64:/etc/firewall# tcpdump -ni nflog:11 -ttt
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on nflog:11, link-type NFLOG (Linux netfilter log messages), snapshot le
ngth 262144 bytes
 00:00:00.000000 IP 192.168.1.144 > 192.168.1.185: ICMP echo request, id 1, seq 48
, length 40
 00:00:00.000652 IP 192.168.1.144 > 192.168.1.185: ICMP echo request, id 1, seq 49
, length 40
 00:00:02.015405 IP 192.168.1.144 > 192.168.1.185: ICMP echo request, id 1, seq 50
, length 40
 00:00:01.023937 IP 192.168.1.144 > 192.168.1.185: ICMP echo request, id 1, seq 51
, length 40
```

```
tcpdump -ni nflog:11 -ttt
```

Trên Windows: gửi 4 gói ICMP:

```
C:\Users\nguye>ping 192.168.1.185

Pinging 192.168.1.185 with 32 bytes of data:
Reply from 192.168.1.185: bytes=32 time<1ms TTL=64
Reply from 192.168.1.185: bytes=32 time<1ms TTL=64
Reply from 192.168.1.185: bytes=32 time<1ms TTL=64
Reply from 192.168.1.185: bytes=32 time<1ms TTL=64

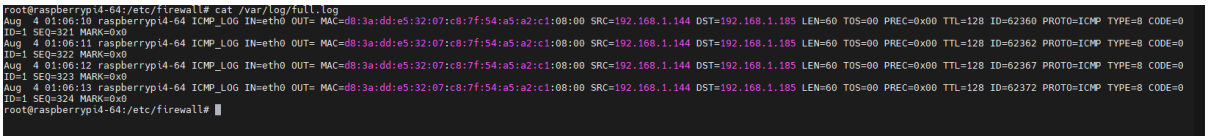
Ping statistics for 192.168.1.185:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 0ms, Maximum = 0ms, Average = 0ms

C:\Users\nguye>
```

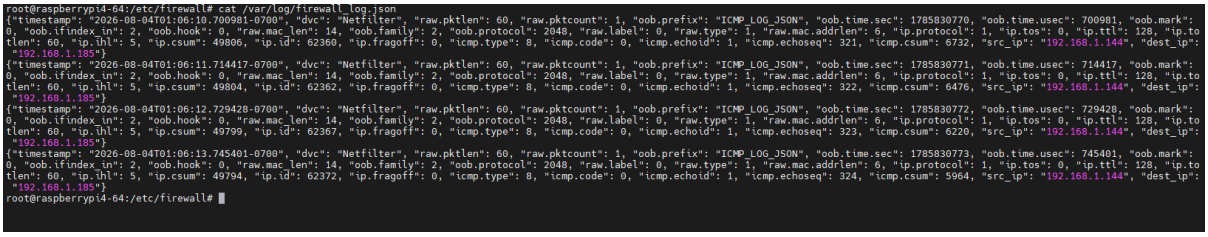
```
ping 192.168.1.185 -n 4
```

4. Kết quả mong đợi

- full.log ghi nhận đủ 4 dòng log, mỗi dòng có: interface (eth0), MAC nguồn/đích, IP nguồn/đích, TTL, PROTO=ICMP TYPE=8 CODE=0.
- `firewall_log.json` ghi nhận đủ 4 record dạng JSON với field tương ứng (`ip.saddr`, `ip.daddr`, `icmp.type=8`, `icmp.code=0`).



Hình 6.1: Trích full.log — ICMP



Hình 6.2: Trích firewall_log.json — ICMP

5. Giải thích / Phân tích log

Log full.log cho thấy traffic đến từ cổng vật lý eth0, là gói tin IPv4 từ MAC nguồn c8:7f:54:a5:a2:c1 đến MAC đích Raspberry Pi (d8:3a:dd:e5:32:07), từ IP nguồn 192.168.1.144 đến IP đích 192.168.1.185, kích thước gói tin 60 bytes, TTL=128 (đặc trưng hệ điều hành Windows), PROTO=ICMP TYPE=8 CODE=0 xác nhận là gói Echo Request (ping).

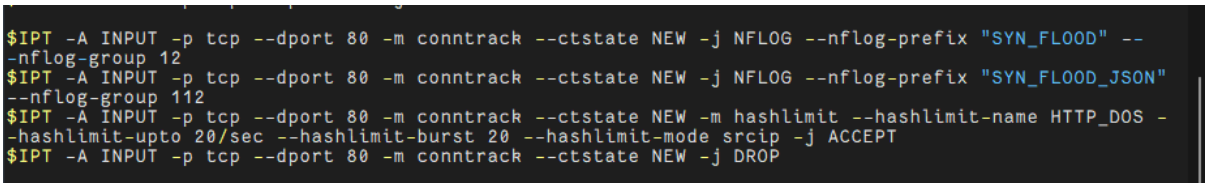
File firewall_log.json ghi nhận tương tự dưới dạng cấu trúc key-value, cho phép hệ thống SIEM/log-analysis đọc và lọc tự động theo từng field mà không cần parse chuỗi text.

6.5.2 TC-W4-02 — Log SYN Flood attack

Mục tiêu: Ghi lại bằng chứng khi có traffic SYN flood vượt ngưỡng hashlimit trên port 80, trước khi bị ACCEPT/DROP bởi rule.

1. Rule iptables/ip6tables bổ sung

```
$IPT -A INPUT -p tcp --dport 80 -m conntrack --ctstate NEW -j NFLOG --nflog-prefix
↪ "SYN_FLOOD" --nflog-group 12
$IPT -A INPUT -p tcp --dport 80 -m conntrack --ctstate NEW -j NFLOG --nflog-prefix
↪ "SYN_FLOOD_JSON" --nflog-group 112
$IPT -A INPUT -p tcp --dport 80 -m conntrack --ctstate NEW -m hashlimit
↪ --hashlimit-name HTTP_DOS \
  --hashlimit-upto 20/sec --hashlimit-burst 20 --hashlimit-mode srcip -j ACCEPT
$IPT -A INPUT -p tcp --dport 80 -m conntrack --ctstate NEW -j DROP
```

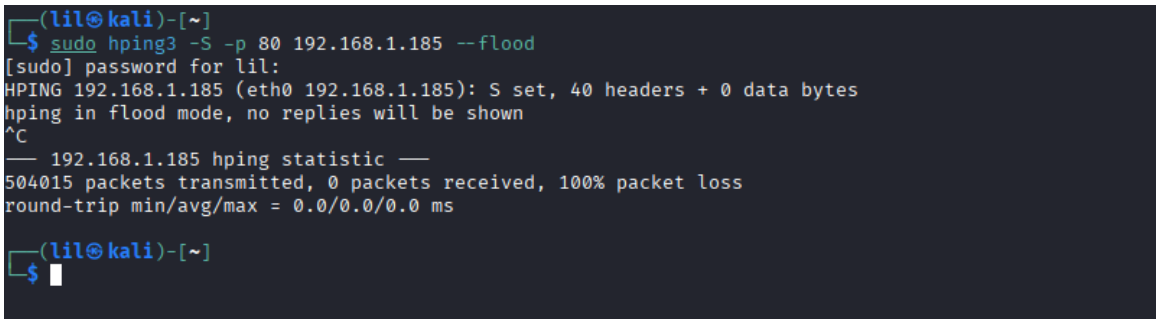


2. Stack ulogd2 tương ứng (/etc/ulogd.conf)

```
stack=log3:NFLOG,base3:BASE,ifi3:IFINDEX,hwhdr3:HWHDR,ip3:IP2STR,print3:PRINTPKT,
↪ emu3:LOGEMU
stack=log3j:NFLOG,base3j:BASE,ifi3j:IFINDEX,hwhdr3j:HWHDR,ip3j:IP2STR,json3j:JSON
[log3] group=12
[log3j] group=112
```

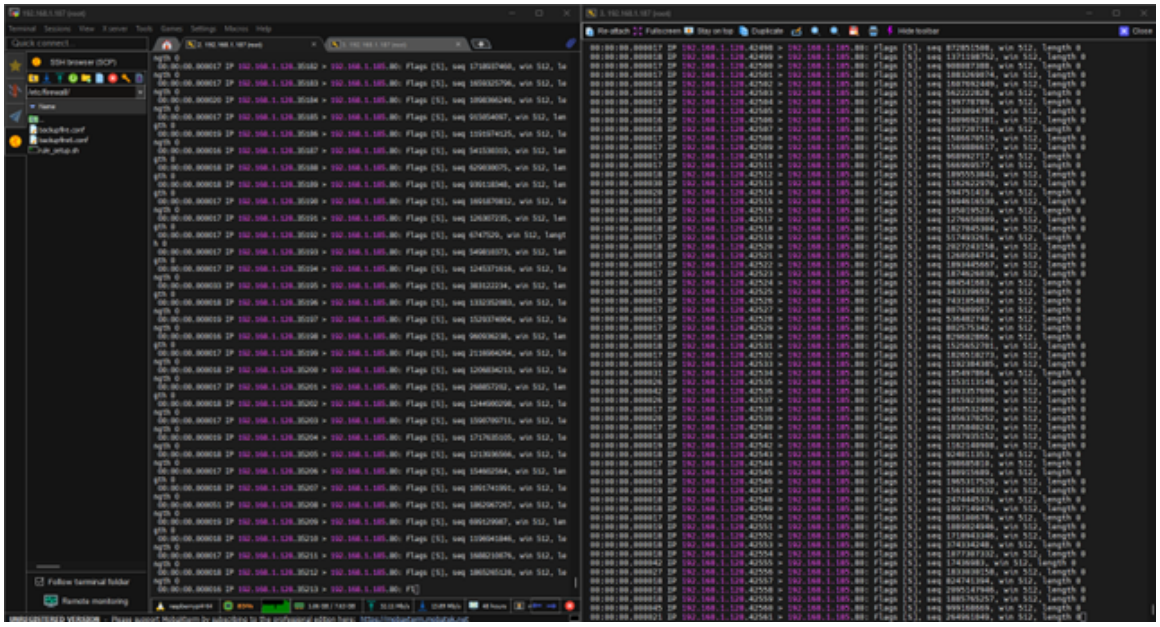
3. Các bước thực hiện

Trên Windows (hoặc máy có hping3): sinh SYN flood tới port 80:




```
hping3 -S -p 80 192.168.1.185 --flood
```

Chạy trong khoảng 5 giây rồi dừng. Xác nhận kênh NFLOG có dữ liệu (chạy song song trước khi sinh traffic):



`tcpdump -ni nflog:12 -ttt và tcpdump -ni nflog:112 -ttt`

```
2
 500K   20M NFLOG      tcp  --  *      *       0.0.0.0/0            0.0.0.0/0
0 ctstate NEW nflog-prefix  SYN_FLOOD nflog-group 12
 500K   20M NFLOG      tcp  --  *      *       0.0.0.0/0            0.0.0.0/0
0 ctstate NEW nflog-prefix  SYN_FLOOD_JSON nflog-group 112
  375 15000 ACCEPT     tcp  --  *      *       0.0.0.0/0            0.0.0.0/0
0 ctstate NEW limit: up to 20/sec burst 20 mode srcip
 500K   20M DROP       tcp  --  *      *       0.0.0.0/0            0.0.0.0/0
0 ctstate NEW
    0     0 ACCEPT     tcp  --  *      *       0.0.0.0/0            0.0.0.0/0
```

iptables -nvL tăng nhanh.

4. Kết quả mong đợi (Pass Criteria)

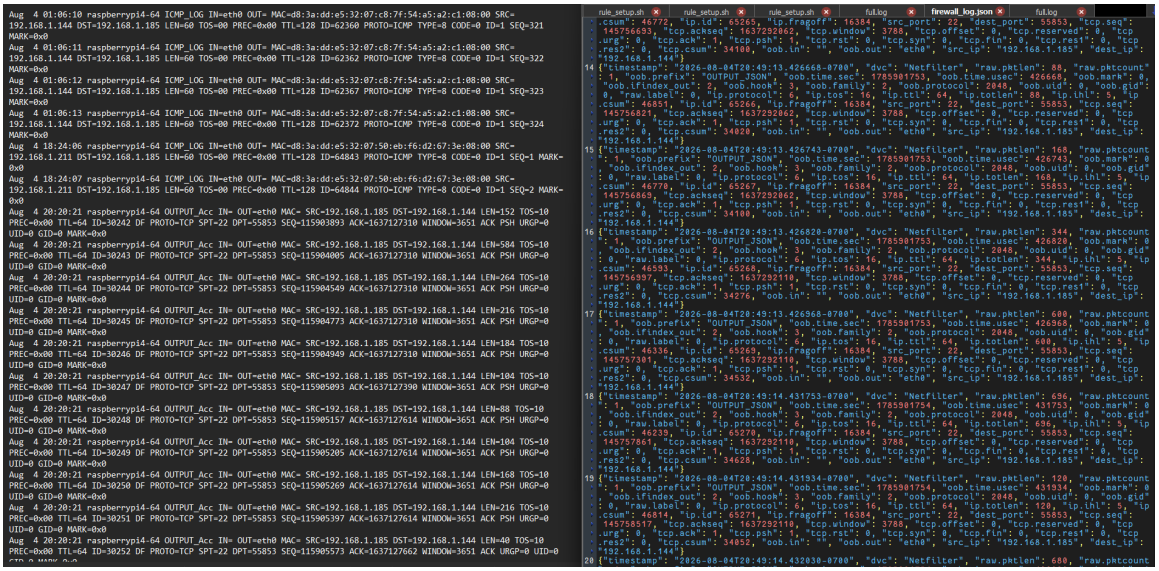
- full.log ghi nhận số lượng lớn dòng SYN_FLOOD trong thời gian ngắn, cùng một IP nguồn.
- firewall_log.json ghi nhận số record tương ứng với tcp.flags chứa SYN.
- Sau khi vượt ngưỡng 20/sec, các gói SYN tiếp theo vẫn được NFLOG log lại (vì NFLOG đặt trước hashlimit) dù bị DROP ở rule sau.

5. Giải thích / Phân tích log

Log cho thấy Raspberry Pi (192.168.1.185) đang có lưu lượng TCP SSH đến 192.168.1.144, với source port 22 → destination port 55853, trạng thái ACK/PSH, và các gói đều được firewall OUTPUT_Acc (ACCEPT) cho phép đi ra qua eth0. Ngoài ra có một gói ICMP Echo Request từ 192.168.1.211 → 192.168.1.185. Nhìn chung, log thể hiện kết nối SSH đang hoạt động giữa Pi và 192.168.1.144, đồng thời Pi nhận ping từ 192.168.1.211.

6.5.3 TC-W4-03 — Log Broadcast IPv4

Mục tiêu: Ghi lại bằng chứng cho traffic Broadcast IPv4 hợp lệ (đích 192.168.1.255), nối lại với test case Broadcast đã xây ở tuần 3.



Hình 6.3: Kết quả log SYN Flood

1. Rule iptables/ip6tables bổ sung

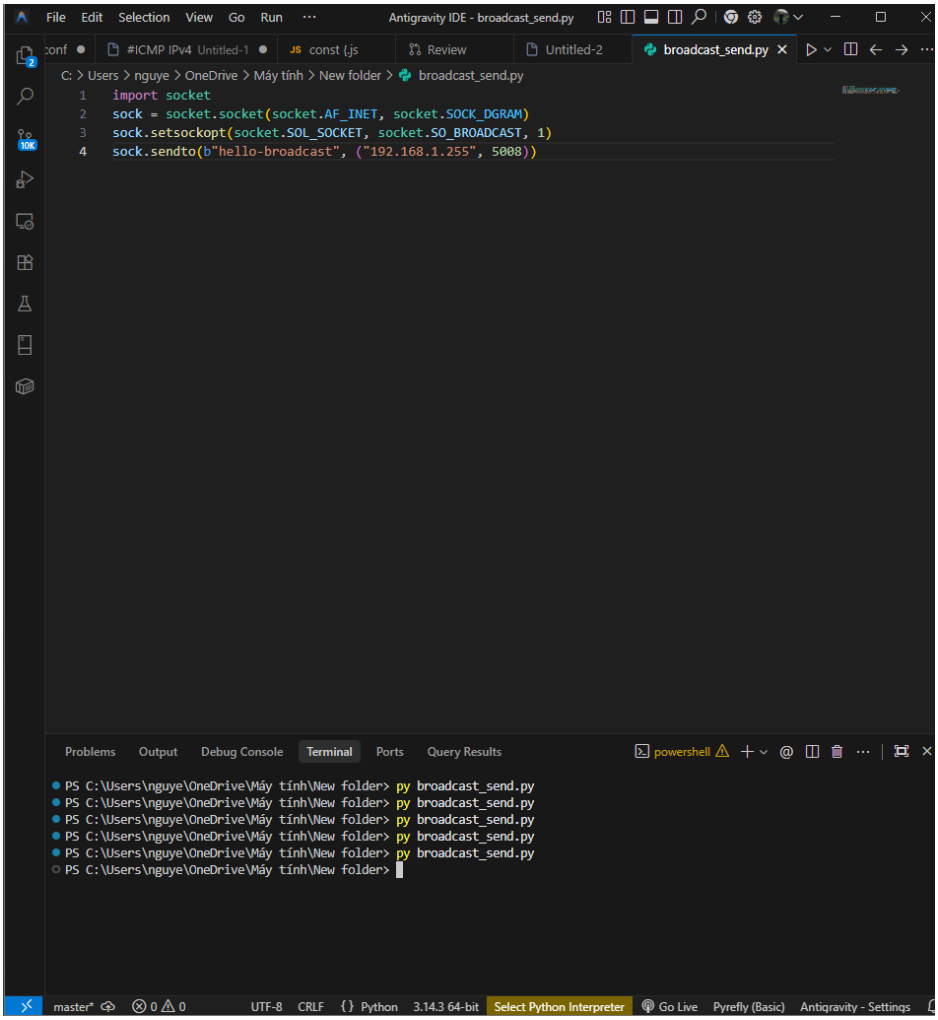
```
$IPT -t mangle -A PREROUTING -d 192.168.1.255 -p udp --dport 5008 -j NFLOG
↪  --nflog-prefix ”BROADCAST” --nflog-group 13
$IPT -t mangle -A PREROUTING -d 192.168.1.255 -p udp --dport 5008 -j NFLOG
↪  --nflog-prefix ”BROADCAST_JSON” --nflog-group 113
$IPT -t mangle -A PREROUTING -d 192.168.1.255 -p udp --dport 5008 -j ACCEPT
```

```
#Broadcast
$IPT -t mangle -A PREROUTING -d 192.168.1.255 -p udp --dport 5008 -j NFLOG --nflog-prefix "BROADCAST" --nflog-group 13
$IPT -t mangle -A PREROUTING -d 192.168.1.255 -p udp --dport 5008 -j NFLOG --nflog-prefix "BROADCAST_JSON" --nflog-group 113
$IPT -t mangle -A PREROUTING -d 192.168.1.255 -p udp --dport 5008 -j ACCEPT
```

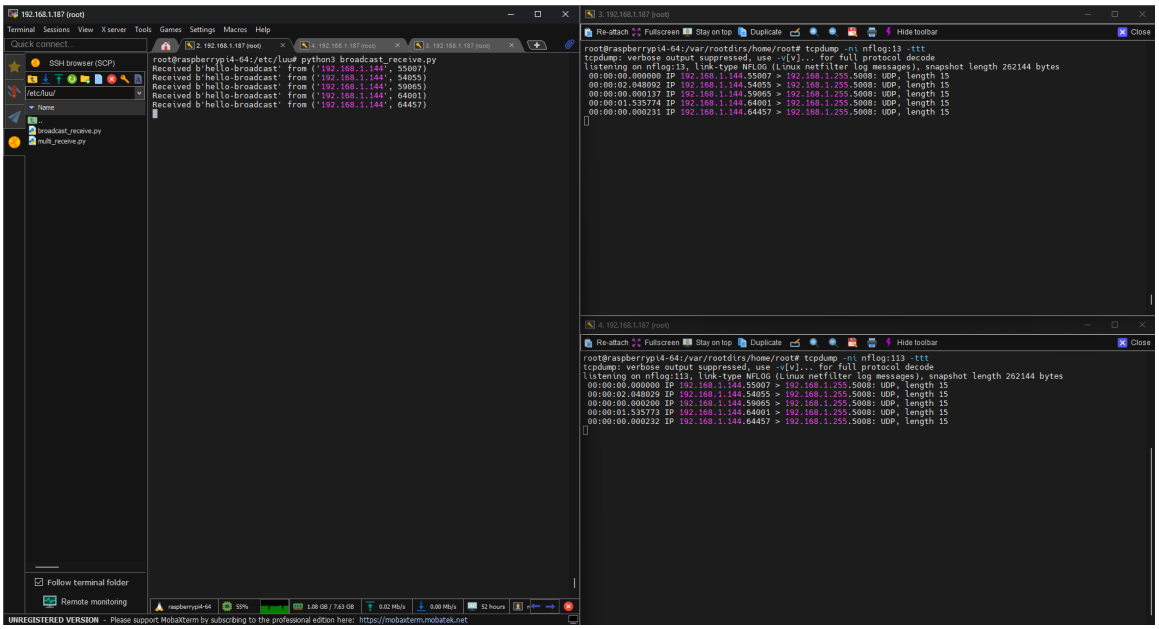
2. Stack ulogd2 tương ứng (/etc/ulogd.conf)

```
stack=log4:NFLOG,base4:BASE,ifi4:IFINDEX,hwhdr4:HWHDR,ip4:IP2STR,print4:PRINTPKT,
↪  emu4:LOGEMU
stack=log4j:NFLOG,base4j:BASE,ifi4j:IFINDEX,hwhdr4j:HWHDR,ip4j:IP2STR,json4j:JSON
[log4] group=13
[log4j] group=113
```

3. Các bước thực hiện



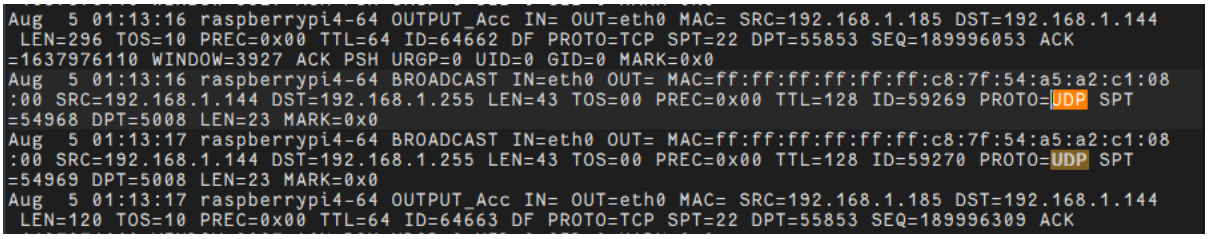
Trên Windows: gửi UDP broadcast tới 192.168.1.255. Xác nhận kênh NFLOG có dữ liệu:



`tcpdump -ni nflog:13 -ttt` và `tcpdump -ni nflog:113 -ttt`

4. Kết quả mong đợi (Pass Criteria)

- full.log ghi nhận dòng BROADCAST với ip.daddr = 192.168.1.255, đúng dport 5008.
- firewall_log.json ghi record tương ứng, có thể lọc theo ip.daddr = 192.168.1.255 để thống kê riêng traffic broadcast.



Hình 6.4: Log Broadcast IPv4

5. Giải thích / Phân tích log

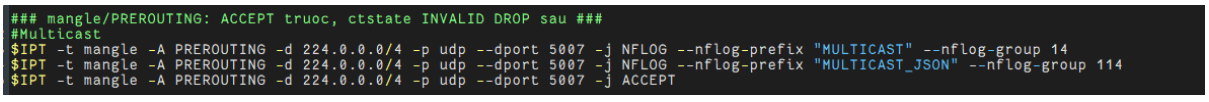
Hình ảnh ghi nhận khả năng giám sát đa nhiệm của hệ thống tường lửa khi bắt thành công hai luồng lưu lượng mạng diễn ra song song. Cụ thể, hệ thống đã tóm gọn các gói tin quảng bá đầu vào (gắn nhãn BROADCAST) sử dụng giao thức UDP, xuất phát từ máy khách 192.168.1.144 nhắm trực tiếp đến địa chỉ MAC và IP Broadcast toàn mạng (ff:ff:ff:ff:ff:ff / 192.168.1.255) tại cổng đích 5008. Xen kẽ với đó, tường lửa cũng kiểm soát chặt chẽ lưu lượng đầu ra (gắn nhãn OUTPUT_Acc), ghi nhận chi tiết luồng dữ liệu phản hồi từ dịch vụ quản trị SSH (TCP, Cổng 22) của Raspberry Pi trả về máy khách.

6.5.4 TC-W4-04 — Log Multicast IPv4

Mục tiêu: Ghi lại bằng chứng cho traffic Multicast IPv4 hợp lệ (nhóm 224.0.0.0/4), nối lại với test case Multicast/IGMP đã xây ở tuần 3.

1. Rule iptables/ip6tables bổ sung

```
$IPT -t mangle -A PREROUTING -d 224.0.0.0/4 -p udp --dport 5007 -j NFLOG
↪ --nflog-prefix "MULTICAST" --nflog-group 14
$IPT -t mangle -A PREROUTING -d 224.0.0.0/4 -p udp --dport 5007 -j NFLOG
↪ --nflog-prefix "MULTICAST_JSON" --nflog-group 114
$IPT -t mangle -A PREROUTING -d 224.0.0.0/4 -p udp --dport 5007 -j ACCEPT
```



2. Stack ulogd2 tương ứng (/etc/ulogd.conf)

```
stack=log5:NFLOG,base5:BASE,ifi5:IFINDEX,hwhdr5:HWHDR,ip5:IP2STR,print5:PRINTPKT,
↪ emu5:LOGEMU
stack=log5j:NFLOG,base5j:BASE,ifi5j:IFINDEX,hwhdr5j:HWHDR,ip5j:IP2STR,json5j:JSON
[log5] group=14
```

```
[log5j] group=114
```

3. Các bước thực hiện

```
PS C:\Users\nguye\OneDrive\Máy tính\New folder>
PS C:\Users\nguye\OneDrive\Máy tính\New folder> py multi_send.py
PS C:\Users\nguye\OneDrive\Máy tính\New folder> py multi_send.py
PS C:\Users\nguye\OneDrive\Máy tính\New folder> py multi_send.py
PS C:\Users\nguye\OneDrive\Máy tính\New folder>
```

Trên Windows: gửi UDP tới nhóm multicast (vd 224.1.1.1) đã join ở tuần 3, dport 5007.

```
root@raspberrypi4-64:/home/root# cd /etc/luu
root@raspberrypi4-64:/etc/luu# python3 multi_receive.py
Raspberry Pi liste Multicast group 224.12.1.1...
Message Group 224.12.1.1 (Win2)
Message Group 224.12.1.1 (Win2)
Message Group 224.12.1.1 (Win2)
Message Group 224.12.1.1 (Win2)
^CTraceback (most recent call last):
  File "/etc/luu/multi_receive.py", line 13, in <module>
    print(sock.recv(1024).decode())
KeyboardInterrupt

root@raspberrypi4-64:/etc/luu#
```

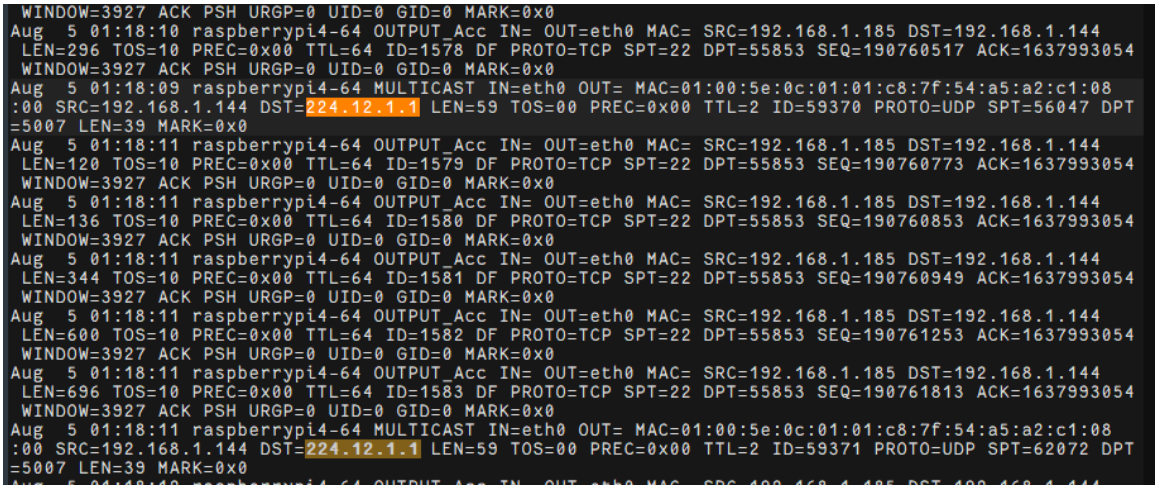
Dùng lại chương trình python gửi/nhận Multicast (IGMP) đã viết ở tuần 3. Xác nhận kênh NFLOG có dữ liệu:

```
root@raspberrypi4-64:/var/rootdirs/home/root# tcpdump -ni nflog:14 -ttt
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on nflog:14, link-type NFLOG (Linux netfilter log messages), snapshot length 262144 bytes
 00:00:00.000000 IP 192.168.1.144.64366 > 224.12.1.1.5007: UDP, length 31
 00:00:02.815818 IP 192.168.1.144.63013 > 224.12.1.1.5007: UDP, length 31
 00:00:01.696032 IP 192.168.1.144.62624 > 224.12.1.1.5007: UDP, length 31
 00:00:00.000155 IP 192.168.1.144.51512 > 224.12.1.1.5007: UDP, length 31
 00:00:01.535780 IP 192.168.1.144.51513 > 224.12.1.1.5007: UDP, length 31
```

```
tcpdump -ni nflog:14 -ttt
```

```
PS C:\Users\nguye\OneDrive\Máy tính\New folder>
PS C:\Users\nguye\OneDrive\Máy tính\New folder>
PS C:\Users\nguye\OneDrive\Máy tính\New folder> py multi_send.py
PS C:\Users\nguye\OneDrive\Máy tính\New folder> py multi_send.py
PS C:\Users\nguye\OneDrive\Máy tính\New folder> py multi_send.py
PS C:\Users\nguye\OneDrive\Máy tính\New folder>
```

4. Kết quả mong đợi (Pass Criteria)



- full.log ghi nhận dòng MULTICAST với ip.daddr nằm trong dải 224.0.0.0/4.

5. Giải thích / Phân tích log

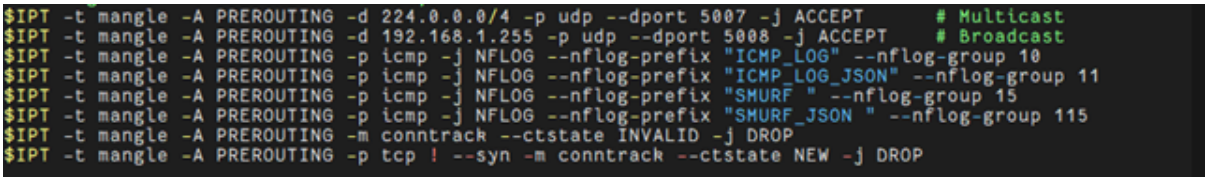
Hình ảnh thể hiện kết quả kiểm thử thành công kịch bản giám sát lưu lượng đa phương (Multicast). Hệ thống tường lửa đã phân luồng và tóm gọn chính xác các gói tin đầu vào mang nhãn MULTICAST qua giao diện eth0. Đặc trưng kỹ thuật được bóc tách rõ nét: gói tin sử dụng giao thức UDP, xuất phát từ IP 192.168.1.144 và nhắm đến dải địa chỉ IP Multicast 224.12.1.1 tại cổng đích 5007.

6.5.5 TC-W4-05 — Log Smurf Attack

Mục tiêu: Ghi lại bằng chứng khi có gói ICMP giả mạo nguồn gửi tới địa chỉ broadcast nhằm khuếch đại tấn công (Smurf attack).

1. Rule iptables/ip6tables bổ sung

```
$IPT -t mangle -A PREROUTING -p icmp -j NFLOG --nflog-prefix "SMURF_CHECK"
↪ --nflog-group 15
$IPT -t mangle -A PREROUTING -p icmp -j NFLOG --nflog-prefix "SMURF_CHECK"
↪ --nflog-group 115
```



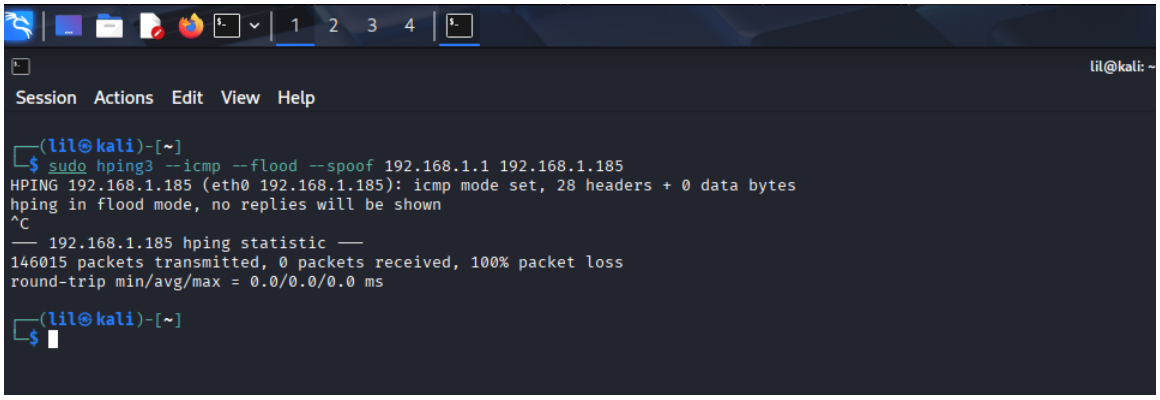
2. Stack ulogd2 tương ứng (/etc/ulogd.conf)

```
stack=log15t:NFLOG,base15t:BASE,ifi15t:IFINDEX,hw15t:HWHDR,ip15t:IP2STR,print15t:
↪ PRINTPKT,emu15:LOGEMU
stack=log15j:NFLOG,base15j:BASE,ifi15j:IFINDEX,hw15j:HWHDR,ip15j:IP2STR,json15:JS
↪ ON
[log15t] group=15
[log15j] group=15
```

Lưu ý: Trong bảng tổng hợp group và cấu hình tổng (mục 6.3–6.4), kênh JSON Smurf được quy hoạch group 115; khi kiểm thử cần đảm bảo stack JSON bind đúng group khớp với rule iptables để tránh xung đột exclusive bind (xem Chương 7).

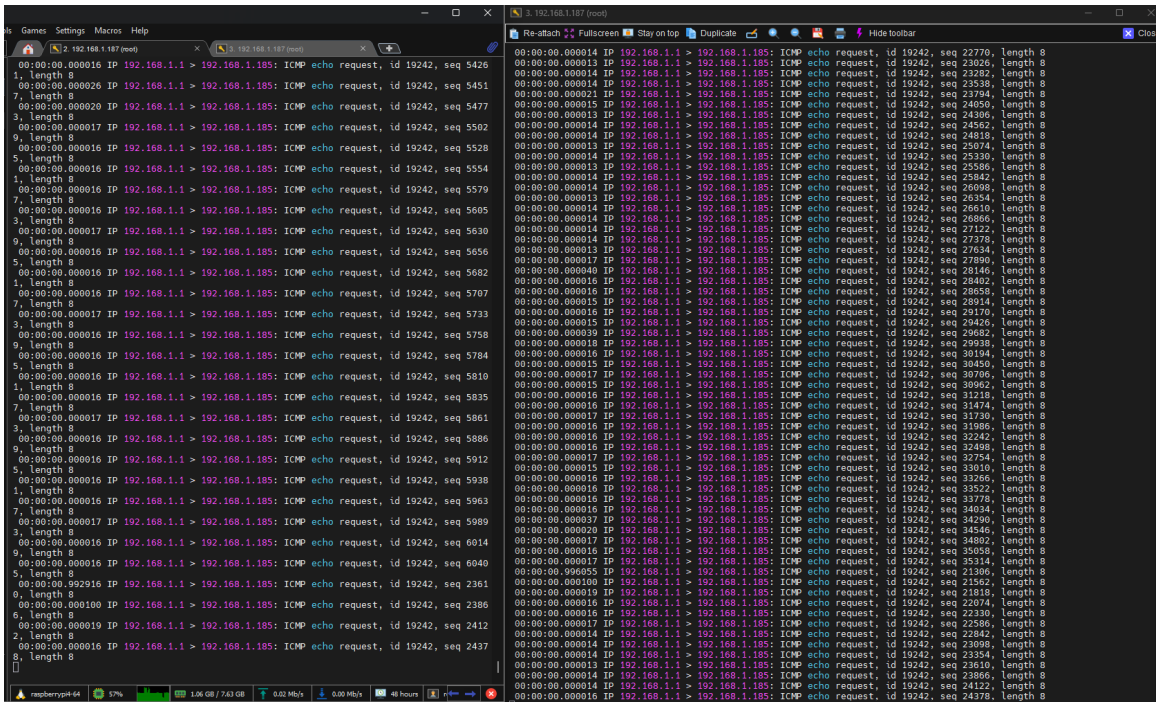
3. Các bước thực hiện

Gửi gói ICMP giả mạo nguồn, flood tới địa chỉ Raspberry:




```
hping3 --icmp --flood --spoof 192.168.1.1 192.168.1.185
```

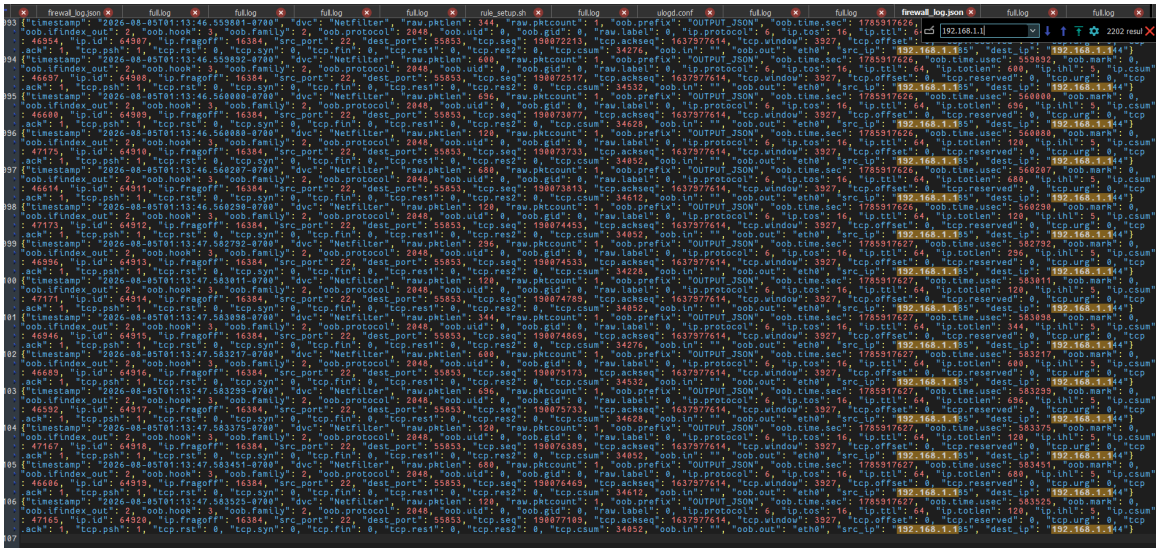
Xác nhận kênh NFLOG có dữ liệu:



```
tcpdump -ni nflog:15 -ttt và tcpdump -ni nflog:115 -ttt
```

4. Kết quả mong đợi (Pass Criteria)

- full.log/firewall_log.json ghi nhận gói ICMP với IP nguồn = 192.168.1.1 (địa chỉ giả mạo), khác với IP thật của máy Windows.
- Không có phản hồi Echo Reply bị khuếch đại ra nhiều máy khác trong mạng (do sysctl đã chặn broadcast echo reply).



Hình 6.5: Log Smurf / JSON liên quan

5. Giải thích / Phân tích log

Hình ảnh minh họa chi tiết kết quả trích xuất log định dạng JSON (Machine-readable) cho kịch bản giám sát lưu lượng đầu ra (Egress Traffic). Hệ thống ghi nhận hàng loạt sự kiện mang nhãn OUTPUT_JSON, phản ánh luồng dữ liệu phản hồi liên tục từ dịch vụ SSH (Cổng nguồn 22, IP 192.168.1.185) gửi về máy khách (192.168.1.144) qua giao diện vật lý eth0. Điểm giá trị nhất của định dạng này nằm ở khả năng bóc tách siêu chuẩn xác: thời gian được ghi nhận chi tiết đến mức micro-giây (timestamp), và đặc biệt các cờ trạng thái TCP phức tạp đã được ulogd2 phân giải thành các biến số Boolean độc lập, chẳng hạn các trường `tcp.ack` và `tcp.psh` nhận giá trị 1.

6.5.6 TC-W4-06 — Audit traffic OUTPUT hợp lệ

Mục tiêu: Ghi lại nhật ký (audit trail) cho traffic hợp lệ rời khỏi Raspberry Pi, phục vụ compliance/audit thay vì chỉ log vi phạm.

1. Rule iptables/ip6tables bổ sung

```
$IPT -A OUTPUT -j NFLOG --nflog-prefix ”OUTPUT_Acc” --nflog-group 16
$IPT -A OUTPUT -j NFLOG --nflog-prefix ”OUTPUT_Acc_JSON” --nflog-group 116
```

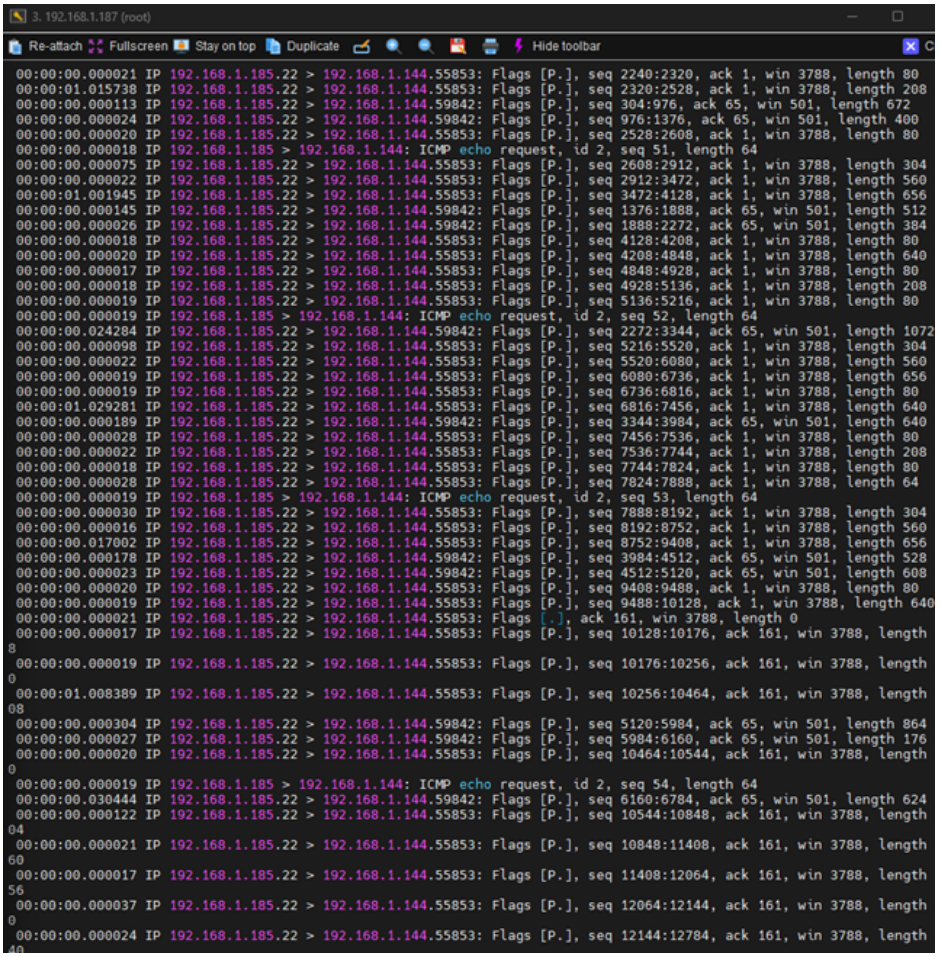
```
$IPT  -P INPUT DROP
$IPT  -P FORWARD DROP
$IPT  -P OUTPUT ACCEPT
$IPT -A OUTPUT -j NFLOG --nflog-prefix "OUTPUT_Acc" --nflog-group 16
$IPT -A OUTPUT -j NFLOG --nflog-prefix "OUTPUT_JSON" --nflog-group 116
$IP6T -P INPUT DROP
$IP6T -P FORWARD DROP
$IP6T -P OUTPUT DROP
$IP6T -A INPUT  -i lo -j ACCEPT
$IP6T -A OUTPUT -o lo -j ACCEPT
```

2. Stack ulogd2 tương ứng (/etc/ulogd.conf)

```
stack=log7:NFLOG,base7:BASE,ifi7:IFINDEX,hwhdr7:HWHDR,ip7:IP2STR,print7:PRINTPKT,
↪  emu7:LOGEMU
stack=log7j:NFLOG,base7j:BASE,ifi7j:IFINDEX,hwhdr7j:HWHDR,ip7j:IP2STR,json7j:JSON
[log7] group=16
[log7j] group=116
```

3. Các bước thực hiện

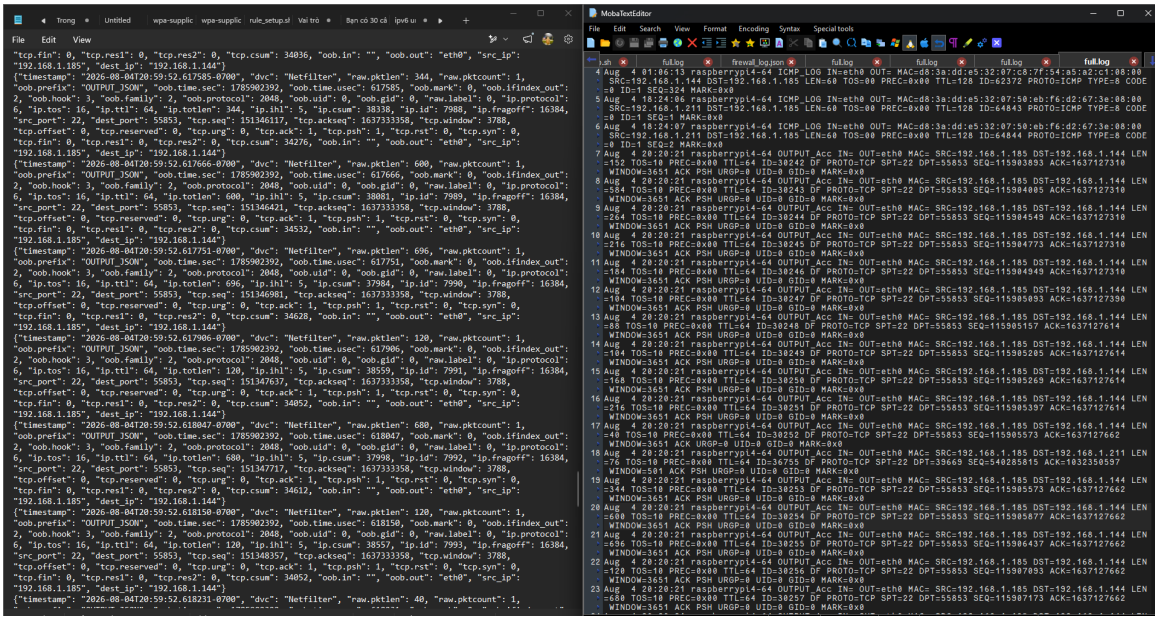
Trên Raspberry: sinh traffic: ping 192.168.1.144. Xác nhận kênh NFLOG có dữ liệu:




```
tcpdump -ni nflog:16 -ttt và tcpdump -ni nflog:116 -ttt
```

4. Kết quả mong đợi

- full.log/firewall_log.json ghi nhận toàn bộ traffic OUTPUT: ICMP reply, TCP, UDP… không chỉ một loại.
- Số lượng dòng log OUTPUT_AUDIT tỉ lệ thuận với số traffic thực tế gửi đi trong khoảng thời gian test.



Hình 6.6: Log audit OUTPUT

5. Giải thích / Phân tích log

Thiết bị đã tóm gọn đồng thời hai sự kiện mạng riêng biệt: luồng Ping thăm dò đầu vào (gắn nhãn ICMP_LOG từ IP 192.168.1.211) và chuỗi dữ liệu phản hồi quản trị đầu ra (gắn nhãn OUTPUT_Acc / OUTPUT_JSON từ dịch vụ SSH của máy chủ trả về thiết bị khách 192.168.1.144).

7 XỬ LÝ SỰ CỐ VÀ QUYẾT ĐỊNH THIẾT KẾ

Trong quá trình tích hợp và kiểm thử hệ thống ulogd2 cùng tường lửa iptables/ip6tables, nhóm đã phát hiện và xử lý dứt điểm các điểm nghẽn kỹ thuật sau.

7.1 Xung đột cơ chế lắng nghe của NFLOG (Lỗi rỗng file JSON & Unable to bind)

Hiện tượng:

File firewall_log.json liên tục bị rỗng đối với các kịch bản SYN flood, LAND, Smurf, OUTPUT, IPv6 (dù full.log vẫn có dữ liệu). Đôi khi hệ thống báo lỗi `unable to bind to log group X` và crash tiến trình.

Phân tích nguyên nhân:

Kernel quản lý các group NFLOG theo nguyên tắc Độc quyền Lắng nghe (Exclusive Subscription). Một group chỉ cho phép duy nhất một tiến trình (ulogd stack) bind vào tại một thời điểm. Tuy nhiên, ở chiều ngược lại, cơ chế Phát (Publish) lại không bị giới hạn — nhiều rule iptables có thể cùng đẩy log vào một group. Việc gán chung 1 group cho 2 stack (TEXT và JSON) đã gây ra xung đột chiếm quyền, khiến stack JSON bị văng.

Cách khắc phục:

Quy hoạch lại kiến trúc phân kênh. Mỗi loại traffic sử dụng 2 rule iptables riêng biệt, đẩy lên 2 group độc lập (Quy ước: Group_JSON = Group_TEXT + 100). Đồng thời, dọn dẹp các tiến trình kẹt ngầm bằng killall -9 ulogd tcpdump.

7.2 Thiếu trường dữ liệu phần cứng (Lỗi cannot find key `oob.in`)

Hiện tượng:

Khởi động ulogd báo lỗi thiếu key và từ chối dựng stack có chứa plugin PRINTPKT.

Phân tích nguyên nhân:

Module biến đổi văn bản PRINTPKT yêu cầu định danh giao diện mạng đầu vào (oob.in), nhưng các plugin cơ sở như BASE hay IP2STR không tự tạo trường này. Việc stack JSON thiếu thông tin phần cứng cũng là nguyên nhân gây crash ngầm tiến trình.

Cách khắc phục:

Chèn thêm plugin IFINDEX và HWHDR vào chuỗi cấu hình (đặt ngay trước IP2STR) trên cả hai stack TEXT và JSON để bổ sung siêu dữ liệu (metadata) Layer 2.

7.3 Điều chỉnh kịch bản kiểm thử: thay Port Scanning/LAND bằng Broadcast/Multicast

Quá trình kiểm thử hai kịch bản gốc gặp các vướng mắc đi sâu vào nhân hệ điều hành (Kernel-level):

- Port Scanning: Rule iptables bắt cờ –tcp-flags SYN,ACK,FIN,RST RST chỉ khớp với gói RST thuần.

Trong khi đó, trình quét nmap mặc định sử dụng cơ chế SYN Scan (Half-open), dẫn đến việc gói tin không bao giờ khớp rule.

- LAND Attack: Kịch bản giả mạo IP nguồn bị cơ chế chống giả mạo của Linux Kernel (rp_filter – Martian packet filtering) tự động lọc bỏ ở Tầng Định tuyến (Routing Layer) trước khi gói tin kịp đi vào chuỗi filter/INPUT của Netfilter.

Quyết định thiết kế (Pivot):

Thay thế bằng kịch bản Broadcast IPv4 và Multicast IPv4. Quyết định này giúp tái sử dụng các rule ACCEPT từ tuần 3, kết nối liền mạch kiến thức (đủ 3 chuẩn giao tiếp Unicast/Broadcast/Multicast), đồng thời loại trừ được các biến số phức tạp từ Kernel làm nhiễu quá trình kiểm thử ulogd2.

Lưu ý chuyên môn:

Đối với các tấn công giả mạo (Spoofing), các rule ghi log (NFLOG) bắt buộc phải đặt tại chain mangle/PREROUTING để “đón đầu” trước khi Kernel xử lý định tuyến.

8 KẾT LUẬN VÀ BÀI HỌC KINH NGHIỆM

8.1 Tổng kết thành quả triển khai

Hệ thống đã tích hợp thành công kiến trúc giám sát ulogd2 (từ package nguyên bản) trên nền tảng WebOS (Raspberry Pi 4).

- Khắc phục triệt để các lỗi vận hành (thiếu siêu dữ liệu phần cứng, crash ngầm do xung đột kênh).
- Mở rộng phạm vi giám sát từ baseline ICMP sang hệ sinh thái toàn diện: SYN Flood, Broadcast, Multicast, Smurf, và đặc biệt là kiểm toán thành công lưu lượng hợp lệ đầu ra (OUTPUT Egress Traffic).
- Xuất log song song hai định dạng full.log (LOGEMU) và firewall_log.json (JSON) theo yêu cầu VanFit / SIEM.

8.2 Nguyên lý cốt lõi rút ra (Core Principles)

Nguyên tắc vận hành mạng sống còn của NFLOG: Mỗi Group chỉ cung cấp duy nhất 1 luồng lắng nghe độc quyền. Để xuất log đa định dạng (Syslog Text + JSON) cho cùng một loại traffic, bắt buộc phải sử dụng kiến trúc Phân mảnh Luồng dữ liệu (Traffic Splitting) bằng 2 rule iptables đẩy lên 2 group song song.

8.3 Nhận xét cá nhân và định hướng phát triển

Thách thức lớn nhất:

Vượt qua sự thiếu nhất quán giữa lý thuyết mạng và cách Kernel Linux thực tế xử lý gói tin (như cơ chế rp_filter "ngầm" vô hiệu hóa LAND Attack, hay giới hạn binding của nfnetlink_log). Điều này đòi hỏi kỹ năng tư duy ngược (reverse thinking) để gỡ lỗi từ tầng ứng dụng xuống tận tầng Kernel.

Kiến thức đạt được:

Nắm vững kiến trúc đa luồng của hệ thống phân tích log tập trung; hiểu sâu về vòng đời của gói tin đi qua Netfilter (PREROUTING → INPUT/OUTPUT).

Định hướng tối ưu hóa (Future Works):

- Triển khai cấu hình logrotate để chống tràn ổ cứng (Disk exhaustion) khi hệ thống hoạt động thực tế.
- Tích hợp thêm module nfacct nhằm đối chiếu chính xác số liệu băng thông (Byte/Packet counters) giữa Log và Thực tế, phục vụ lập profile hành vi theo chuẩn Zero Trust.

TÀI LIỆU THAM KHẢO

[1] 5_1_iptables__netfilter.pdf (Tài liệu nền tảng Netfilter).

[2] 5_4_NfacctNflog.pdf (Kiến trúc Log và Accounting).

[3] 5_6_UlogdLogrotate.pdf (Quản lý và xoay vòng Log).

[4] 12_2_Task_and_Test_case_Example_v1.pdf (Quy chuẩn định dạng và kịch bản test).

[5] iptables.sh (Source code Firewall – Bản cập nhật Tuần 3-4).

[6] /etc/ulogd.conf (Source code Cấu hình ulogd2 – Bản mở rộng Tuần 4).

[7] Netfilter Project – ulogd2 documentation: https://www.netfilter.org/projects/ulogd/

[8] OpenEmbedded meta-networking recipe ulogd2 (Yocto/OpenEmbedded).